3D-SCPTOP: model, reformulation and solution method

Working document – formulation, algorithms and computational study

October 21, 2026

Contents

1 Mathematical Model **1**
1.1 Symmetry breaking, slot-set sizing and data assumptions 8

2 Dantzig–Wolfe reformulation and branch and price **9**
2.1 The decomposition 10
2.2 Columns 12
2.3 The master problem 13
2.4 The restricted master problem and the reduced cost 14
2.5 Lower bounds 15
2.6 The pricing problems 17
2.7 Initial columns 23
2.8 Relax and fix 24
2.8.1 Flow-first relax and fix 25
2.8.2 Horizon-walk relax and fix 26
2.9 The fleet tree 27
2.10 The construction under 3D loading 30
2.11 Two corrections to the compact model 32

3 Computational Experiments **32**
3.1 Protocol and declared budgets 33
3.2 Instance generation 35
3.3 Methods compared, and which procedure produced which table 36
3.4 The volumetric model against the compact SCPTOP formulation 37
3.5 The 3D problem 39
3.6 Against the procedure in use 48
3.7 Is a volumetric capacity adequate? 49
3.8 What the bounds and the claims rest on 52

---

1 Mathematical Model

The 3D-SCPTOP formulation integrates stack-based truck-loading constraints with procurement transport and inventory decisions. It departs from a standard 3D container-loading formulation in two related ways. First, the 3D placement of individual RTIs inside the truck is replaced by a 2D placement of variable-composition *stacks* on the truck floor, with per-stack

height and weight caps. Second, the per-period decision of which product fills each slot is retained, but the crush-limit cap is evaluated slot-by-slot using the period's actual product mix. This yields a tighter per-stack weight bound than a worst-case-per-RTI-type abstraction, while eliminating vertical-stability constraints altogether, since stacks are self-supporting by construction.

The model is written over *trucks*. A dispatched truck designs its own layout: the stack types, orientations, positions and layer compositions of its slots are variables of that truck and of that period, and the load it carries is assigned to those slots in the same period. There is no catalogue of loading configurations, and no variable that assigns a truck to an entry of one. This is a modelling decision and not a presentational one, and it is the decision Section 2 rests on: every row below carries either the triple (g, k, t) or none of it, so the model is block-angular over $G \times K \times T$ and the pricing problem of the Dantzig–Wolfe reformulation is obtained by deleting that quantifier and nothing else. Section 2.1 performs exactly that deletion, row by row.

Notational conventions. Throughout this section, $g \in G = \{s, p\}$ indexes nodes (supplier and plant), $n \in N$ products, $r \in R$ RTI types, $h \in H$ stack types, $i, j \in I$ stack-slot positions on a truck floor, $k \in K$ trucks and $t \in T$ planning periods. Each product travels in exactly one RTI type, written $\rho(n)$, and $M(r) = \{n \in N : \rho(n) = r\}$ is the family of products that RTI type r carries. Variables whose direction is determined by definition omit the g subscript: V_{kint} denotes full RTIs in slot i of a plant-bound truck, and V^{e}_{krt} folded RTIs on a supplier-bound truck. Table 1 collects the complete nomenclature.

Table 1: Nomenclature of 3D-SCPTOP (truck-indexed stack-based formulation)

Indices and sets	
G	Supplier and plant nodes, $G = \{s, p\}$; $g = s$ is the folded direction (plant to supplier), $g = p$ the full one.
N	Products, $n = 1, \ldots, \lvert N \rvert$.
R	RTI types, $r = 1, \ldots, \lvert R \rvert$.
H	Stack types, $h = 1, \ldots, \lvert H \rvert$.
I	Stack-slot indices on a truck floor, $i = 1, \ldots, \lvert I \rvert$ (see (48)).
K	Trucks available per direction and period, $k = 1, \ldots, \lvert K \rvert$.
T	Planning periods, $t = 1, \ldots, \lvert T \rvert$.
$\rho(n)$	The RTI type in which product n travels.
$M(r)$	Products carried by RTI type r, $M(r) = \{n : \rho(n) = r\} \subseteq N$.
$R(h)$	RTI types compatible with stack type h, $R(h) \subseteq R$.
Parameters	
cK	Cost of dispatching one truck.
lt	Lead transport time between plant and supplier (and vice versa), $lt \geq 1$.
i_{gn}	Initial inventory of $n \in N$ at $g \in G$ (full RTIs).
ie_{gr}	Initial inventory of folded RTIs $r \in R$ at $g \in G$.
ic_{gn}	Inventory holding cost of $n \in N$ at $g \in G$.
d_{gnt}	Production plan for $n \in N$ at $g \in G$ in $t \in T$: filled at s, consumed at p.
a_{nt}	Pipeline arrivals of full RTIs of product n at p in $t \in \{1, \ldots, lt\}$.
ae_{rt}	Pipeline arrivals of folded RTIs of type r at s in $t \in \{1, \ldots, lt\}$.
ss_{gnt}	Coverage stock of n at g in t (full RTIs).
sse_{gtr}	Coverage stock of folded RTIs r at g in t.
w_n	Weight of a full RTI carrying product n.

w_r^{f}	Weight of a folded RTI of type r.
pK^w	Truck payload.
$W^{\max}$	Per-stack crush-limit weight cap: the *total* weight of one stack, summed over all its layers, not the weight borne by its bottom RTI alone.
pK, qK, rK	Truck floor length, floor width, and inner height.
Γ_K	Truck volume, $\Gamma_K = pK \cdot qK \cdot rK$.
α	Minimum truck utilisation, required on whichever of volume or payload binds ($0 < \alpha \leq 1$).
p_h, q_h	Footprint length and width of stack type h.
r_{gr}	Height of one RTI of type r at node g (full at p, folded at s).
γ_{gr}	Volume of one RTI of type r in direction g.
n_{hr}	RTIs of type $r \in R(h)$ per layer of stack type h.
ω_{gr}	Weight per RTI used by the layer-capacity form of the utilisation floor: $\omega_{sr} = w_r^{\mathrm{f}}$ and $\omega_{pr} = \max_{n \in M(r)} w_n$.
λ^K, λ^I	Weights of the transport and inventory objectives; both 1 throughout this paper.
M^x, M^y	Big-M constants of the non-overlap rows, $M^x = pK$ and $M^y = qK$.
$\bar{u}_n^V$	Most full RTIs of n one stack can carry, $\bar{u}_n^V = \lfloor W^{\max}/w_n \rfloor$.
$\bar{u}_r^{\mathrm{e}}$	Most folded RTIs of r one truck can carry, $\bar{u}_r^{\mathrm{e}} = \lfloor pK^w/w_r^{\mathrm{f}} \rfloor$.
Decision variables – the truck and its layout	
Z_{gkt}	$\in \{0,1\}$. Equals 1 iff truck k of direction g in period t is configured, i.e. carries a layout.
Y_{gkt}	$\in \{0,1\}$. Equals 1 iff truck k is dispatched in direction g in period t.
θ_{gkt}	$\in \{0,1\}$. Selects the resource on which the utilisation floor of that truck is required: volume if 1, payload if 0.
X_{gihkt}	$\in \{0,1\}$. Equals 1 iff slot i of that truck holds a stack of type h.
L_{gihktr}	$\in \mathbb{Z}_{\geq 0}$. Layers of RTI type $r \in R(h)$ in slot i of that truck.
LX_{gihkt}	$\in \{0,1\}$. Equals 1 iff stack h's length lies along the x-axis in slot i.
LY_{gihkt}	$\in \{0,1\}$. Equals 1 iff stack h's length lies along the y-axis in slot i.
PX_{gikt}, PY_{gikt}	≥ 0. Bottom-left coordinates of slot i on the truck floor.
A_{gijkt}	$\in \{0,1\}$. Equals 1 iff slot i is to the left of slot j.
B_{gijkt}	$\in \{0,1\}$. Equals 1 iff slot i is to the right of slot j.
C_{gijkt}	$\in \{0,1\}$. Equals 1 iff slot i is behind slot j.
D_{gijkt}	$\in \{0,1\}$. Equals 1 iff slot i is in front of slot j.
Decision variables – load and inventory	
V_{kint}	$\in \mathbb{Z}_{\geq 0}$. Full RTIs of product n in slot i of plant-bound truck k in period t.
V_{krt}^{e}	$\in \mathbb{Z}_{\geq 0}$. Folded RTIs of type r on supplier-bound truck k in period t.
I_{gnt}	$\in \mathbb{Z}_{\geq 0}$. End-of-period inventory of n at g in t (full RTIs).
Ie_{gtr}	$\in \mathbb{Z}_{\geq 0}$. End-of-period inventory of folded RTIs r at g in t.
Derived quantities (abbreviations, not constraints; see (1)–(5))	
$\overline{V}_{nt}$	Full RTIs of n delivered to the plant in t.
$\overline{V^{\mathrm{e}}}_{rt}$	Folded RTIs of r returned to the supplier in t.
$\overline{V}_{knt}$	Full RTIs of n on plant-bound truck k in t.
$\widehat{X}_{gikt}$	Equals 1 iff slot i of that truck is used.
$\overline{L}_{ghktr}$	Layers of RTI type r under stack type h on that truck, over all slots.

Derived quantities

Five sums recur, and naming them is what keeps the model readable and what makes its structure visible. They are *abbreviations*, not constraints: each is a linear expression in the decision variables above, substituted wherever it appears.

$$\overline{V}_{nt} = \sum_{k\in K}\sum_{i\in I} V_{kint} \qquad \forall n \in N,\ t \in T \tag{1}$$

$$\overline{V^{\mathrm{e}}}_{rt} = \sum_{k\in K} V^{\mathrm{e}}_{krt} \qquad \forall r \in R,\ t \in T \tag{2}$$

$$\overline{V}_{knt} = \sum_{i\in I} V_{kint} \qquad \forall k \in K,\ n \in N,\ t \in T \tag{3}$$

$$\widehat{X}_{gikt} = \sum_{h\in H} X_{gihkt} \qquad \forall g \in G,\ i \in I,\ k \in K,\ t \in T \tag{4}$$

$$\overline{L}_{ghktr} = \sum_{i\in I} L_{gihktr} \qquad \forall g \in G,\ h \in H,\ k \in K,\ t \in T,\ r \in R(h) \tag{5}$$

The first two are the hinge of the whole paper. The inventory block reads $\overline{V}_{nt}$ and $\overline{V^{\mathrm{e}}}_{rt}$ and nothing else about the trucks: it does not see which truck carried what, nor how the load was placed. Every other row of the model carries a single (g, k, t). The entire coupling between the transport block and the loading block is therefore the two sums (1)–(2) together with the fact that there are at most $|K|$ trucks per direction and period — which is why the master of Section 2.3 needs exactly two definition rows and one count row, and why (1)–(2) reappear there as the only place a column's image enters.

Objectives

The model has two objectives, transport cost and inventory holding cost:

$$\min\ z^K = \sum_{g\in G}\sum_{k\in K}\sum_{t\in T} cK \cdot Y_{gkt} \tag{6}$$

$$\min\ z^I = \sum_{g\in G}\sum_{n\in N}\sum_{t\in T} ic_{gn} \cdot I_{gnt} \tag{7}$$

and they are solved scalarised,

$$\min\ \ \lambda^K z^K + \lambda^I z^I, \tag{8}$$

with $\lambda^K = \lambda^I = 1$ in every run reported in Section 3, so that the reported cost is transport plus holding cost in the same money. Transport cost counts *dispatches*: a truck that is configured but not dispatched costs nothing, and no term of (6)–(7) charges for a layout.

Inventory and flow balance

Full RTIs arrive at the plant lt periods after they are dispatched; inventory itself accumulates one period at a time, and only the arrival is delayed. For the plant,

$$I_{pnt} = I_{p,n,t-1} - d_{pnt} + \overline{V}_{n,t-lt} \qquad \forall n \in N,\ t \geq lt+1 \tag{9}$$

$$I_{pnt} = I_{p,n,t-1} - d_{pnt} + a_{nt} \qquad \forall n \in N,\ t \in \{2,\dots,lt\} \tag{10}$$

$$I_{pnt} = i_{pn} - d_{pnt} + a_{nt} \qquad \forall n \in N,\ t = 1 \tag{11}$$

$$Ie_{ptr} = Ie_{p,t-1,r} + \sum_{n \in M(r)} d_{pnt} - \overline{V^{\mathrm{e}}}_{rt} \qquad \forall r \in R,\ t \geq 2 \tag{12}$$

$$Ie_{ptr} = ie_{pr} + \sum_{n \in M(r)} d_{pnt} - \overline{V^{\mathrm{e}}}_{rt} \qquad \forall r \in R,\ t = 1 \tag{13}$$

Consuming a full RTI at the plant produces a folded one in the same period, which is why the plant's folded balance is driven by d_{pnt}; the folded RTIs leave on the same day they are shipped, so $\overline{V^{\mathrm{e}}}_{rt}$ enters (12) undelayed and arrives at the supplier lt periods later. For the supplier,

$$I_{snt} = I_{s,n,t-1} + d_{snt} - \overline{V}_{nt} \qquad \forall n \in N,\ t \geq 2 \tag{14}$$

$$I_{snt} = i_{sn} + d_{snt} - \overline{V}_{nt} \qquad \forall n \in N,\ t = 1 \tag{15}$$

$$Ie_{str} = Ie_{s,t-1,r} + \overline{V^{\mathrm{e}}}_{r,t-lt} - \sum_{n \in M(r)} d_{snt} \qquad \forall r \in R,\ t \geq lt+1 \tag{16}$$

$$Ie_{str} = Ie_{s,t-1,r} + ae_{rt} - \sum_{n \in M(r)} d_{snt} \qquad \forall r \in R,\ t \in \{2,\dots,lt\} \tag{17}$$

$$Ie_{str} = ie_{sr} + ae_{rt} - \sum_{n \in M(r)} d_{snt} \qquad \forall r \in R,\ t = 1 \tag{18}$$

Coverage stock is enforced at both nodes and on both kinds of stock:

$$I_{gnt} \geq ss_{gnt} \qquad \forall g \in G,\ n \in N,\ t \in T \tag{19}$$

$$Ie_{gtr} \geq sse_{gtr} \qquad \forall g \in G,\ r \in R,\ t \in T \tag{20}$$

These are the only rows of the model that couple two trucks, two periods or two directions, and they do it only through (1)–(2).

The layout of a truck

Everything from here to (45) is written for one truck: one direction g, one truck index k, one period t. At most one stack type occupies a slot, and only a configured truck has slots:

$$\widehat{X}_{gikt} \ \leq\ Z_{gkt} \qquad \forall g \in G,\ i \in I,\ k \in K,\ t \in T. \tag{21}$$

The stack occupying slot i is oriented either with its length along the x-axis or along the y-axis:

$$LX_{gihkt} + LY_{gihkt} \ =\ X_{gihkt} \qquad \forall g \in G,\ h \in H,\ i \in I,\ k \in K,\ t \in T. \tag{22}$$

The total stack height cannot exceed the truck height. The following constraint also forces $L_{gihktr} = 0$ whenever $X_{gihkt} = 0$, since $r_{gr} > 0$:

$$\sum_{r \in R(h)} r_{gr}\, L_{gihktr} \ \leq\ rK \cdot X_{gihkt} \qquad \forall g \in G,\ h \in H,\ i \in I,\ k \in K,\ t \in T. \tag{23}$$

Slots must not overlap. For each pair $i < j$ of slots of the same truck, with $\forall g \in G,\ k \in K,\ t \in T$ understood throughout:

$$PX_{gikt} + \sum_{h \in H} \big(p_h LX_{gihkt} + q_h LY_{gihkt}\big) \le PX_{gjkt} + M^x(1 - A_{gijkt}) \tag{24}$$

$$PX_{gjkt} + \sum_{h \in H} \big(p_h LX_{gjhkt} + q_h LY_{gjhkt}\big) \le PX_{gikt} + M^x(1 - B_{gijkt}) \tag{25}$$

$$PY_{gikt} + \sum_{h \in H} \big(q_h LX_{gihkt} + p_h LY_{gihkt}\big) \le PY_{gjkt} + M^y(1 - C_{gijkt}) \tag{26}$$

$$PY_{gjkt} + \sum_{h \in H} \big(q_h LX_{gjhkt} + p_h LY_{gjhkt}\big) \le PY_{gikt} + M^y(1 - D_{gijkt}) \tag{27}$$

with the cover constraint ensuring that whenever both slots are used at least one relative-position variable is active:

$$A_{gijkt} + B_{gijkt} + C_{gijkt} + D_{gijkt} \ \ge\ \widehat{X}_{gikt} + \widehat{X}_{gjkt} - 1 \qquad \forall g,\ k,\ t,\ i < j. \tag{28}$$

Containment in the truck floor:

$$PX_{gikt} + \sum_{h \in H} \big(p_h LX_{gihkt} + q_h LY_{gihkt}\big) \le pK \qquad \forall g, i, k, t, \tag{29}$$

$$PY_{gikt} + \sum_{h \in H} \big(q_h LX_{gihkt} + p_h LY_{gihkt}\big) \le qK \qquad \forall g, i, k, t. \tag{30}$$

These two rows are unconditional, and the non-overlap rows take the tightened constants $M^x = pK$ and $M^y = qK$, both because the geometry is now the truck's own. An unused slot has $LX_{gihkt} = LY_{gihkt} = 0$ for every h, so (29)–(30) reduce to $PX_{gikt} \le pK$ and $PY_{gikt} \le qK$ there and cut nothing; and given those bounds an inactive disjunct of (24)–(27) is never binding, since the left-hand side is at most pK (respectively qK) and the other slot's origin is non-negative. Enlarging the two constants only weakens the linear relaxation.

Full-truckload on every dispatched truck

Full-truckload transport requires a dispatched truck to be full, but a truck can be full in two different ways: it may run out of space or it may run out of payload. Imposing the requirement on volume alone is not a conservative simplification, it is an infeasible one — filling a fraction α of a 97.9 m^3 trailer with RTIs at 590 $\mathrm{kg/m}^3$ weighs 27.4 t against a 24 t cap, so no truck is dispatchable at all and the instance is empty. Summing the two ratios instead is the opposite error: it accepts a truck that is 35% full by volume and 20% by weight, which is full by neither. We therefore require a dispatched truck to reach a fraction α of *either* resource, using a binary θ_{gkt} to select which one binds. The requirement is stated on the load the truck actually carries, with each commodity's own weight:

$$\sum_{n \in N} \gamma_{p\rho(n)}\, \overline{V}_{knt} \ \ge\ \alpha\, \Gamma_K \big(Y_{pkt} - 1 + \theta_{pkt}\big) \qquad \forall k \in K,\ t \in T \tag{31}$$

$$\sum_{n \in N} w_n\, \overline{V}_{knt} \ \ge\ \alpha\, pK^w \big(Y_{pkt} - \theta_{pkt}\big) \qquad \forall k \in K,\ t \in T \tag{32}$$

$$\sum_{r \in R} \gamma_{sr}\, V^{\mathrm{e}}_{krt} \ \ge\ \alpha\, \Gamma_K \big(Y_{skt} - 1 + \theta_{skt}\big) \qquad \forall k \in K,\ t \in T \tag{33}$$

$$\sum_{r \in R} w^{\mathrm{f}}_r\, V^{\mathrm{e}}_{krt} \ \ge\ \alpha\, pK^w \big(Y_{skt} - \theta_{skt}\big) \qquad \forall k \in K,\ t \in T \tag{34}$$

At $Y_{gkt} = 0$ both rows of a direction are vacuous; at $Y_{gkt} = 1$ exactly one is active and the other is implied, so each pair is the disjunction "full by volume *or* full by payload".

The same disjunction can be written on the layer *capacity* the truck's stacks provide rather than on the load they receive, and in that form it is a valid tightening:

$$\sum_{h\in H}\sum_{r\in R(h)} \gamma_{gr}\, n_{hr}\, \overline{L}_{ghktr} \;\geq\; \alpha\, \Gamma_K \big(Y_{gkt} - 1 + \theta_{gkt}\big) \qquad \forall g \in G,\ k \in K,\ t \in T \tag{35}$$

$$\sum_{h\in H}\sum_{r\in R(h)} \omega_{gr}\, n_{hr}\, \overline{L}_{ghktr} \;\geq\; \alpha\, pK^{w} \big(Y_{gkt} - \theta_{gkt}\big) \qquad \forall g \in G,\ k \in K,\ t \in T \tag{36}$$

Which of the two forms is the *requirement* and which is the consequence matters, and the paper states it once here. The requirement is (31)–(34): full-truckload is a property of the freight that moves, and a truck whose stacks *could* have been filled to the floor but were not is not full. (35)–(36) are implied by it — a slot never receives more RTIs of type r than its layers hold, by (38)–(39) below, and γ_{gr} depends only on r while $\omega_{gr} \geq w_n$ for every $n \in M(r)$ — so they cut nothing and are kept only because they let a solver establish a first feasible layout without first choosing a product assignment. Taken *alone*, in place of the load rows, they would be strictly weaker and would admit trucks that are not full.

The coefficient ω_{pr} in (36) is the *heaviest* product the family can carry, and both of the reasons point the same way. The row is a $\geq$ requirement, so the smaller coefficient is the stronger row: with $\min_{n\in M(r)} w_n$ the row would assert that *every* product assignment of this layout fills the truck, which is not a necessary condition of "this truck is full" and cuts off loads the problem allows, so that the formulation reports an optimum above the true one — in the very comparison where the decomposition is claimed to beat it. With the maximum the row is implied by the load rows, as the previous paragraph needs it to be; with the minimum it is not implied at all. The same principle governs the crush cap (44), which is why that row is written on the mix actually loaded. A per-load row must use the load's own product weights; only a row that does not yet know the product may use a family bound, and then only the bound that keeps it necessary rather than sufficient.

Dispatch, loading and weight

A dispatched truck must be configured:

$$Y_{gkt} \;\leq\; Z_{gkt} \qquad \forall g \in G,\ k \in K,\ t \in T. \tag{37}$$

The converse does not hold, and the two binaries are not the same variable in this section: $Z_{gkt} = 1$ with $Y_{gkt} = 0$ is a truck that carries a layout and no freight. Such a point costs nothing by (6) and delivers nothing by (40)–(41); Section 2.6 returns to it, because in the reformulated block it is exactly the empty column and the two binaries can then be identified.

For plant-bound trucks, full RTIs of type r loaded in slot i cannot exceed the layer capacity of the stack placed in that slot; products sharing the same RTI type partition that slot capacity:

$$\sum_{n\in M(r)} V_{kint} \;\leq\; \sum_{h\,:\,r\in R(h)} n_{hr}\, L_{p,i,h,k,t,r} \qquad \forall k \in K,\ i \in I,\ r \in R,\ t \in T. \tag{38}$$

For supplier-bound trucks, folded RTIs of type r are capped by the aggregate layer capacity of the truck (no slot-level tracking is needed because folded-RTI weight is deterministic in r):

$$V^{\mathrm{e}}_{krt} \;\leq\; \sum_{h\,:\,r\in R(h)} n_{hr}\, \overline{L}_{s,h,k,t,r} \qquad \forall k \in K,\ r \in R,\ t \in T. \tag{39}$$

Both rows are unconditional: the layers they refer to are the truck's own, and a truck with no stacks has zero capacity on the right-hand side.

Capacity is not the same as dispatch, and the two rows above would leave the load of a configured but undispatched truck free. The loads are therefore also gated on the dispatch, at their own analytic bounds rather than at a generic big-M:

$$V_{kint} \le \bar{u}^{V}_{n}\, Y_{pkt} \qquad \forall k \in K,\ i \in I,\ n \in N,\ t \in T \tag{40}$$

$$V^{\mathrm{e}}_{krt} \le \bar{u}^{\mathrm{e}}_{r}\, Y_{skt} \qquad \forall k \in K,\ r \in R,\ t \in T \tag{41}$$

The constants are the tightest valid ones: no slot can hold more than $\bar{u}^{V}_{n} = \lfloor W^{\max}/w_n \rfloor$ full RTIs of n without breaching the crush cap (44), and no truck more than $\bar{u}^{\mathrm{e}}_{r} = \lfloor pK^{w}/w^{\mathrm{f}}_{r} \rfloor$ folded RTIs of r without breaching the payload (43), so neither row cuts a feasible load and both are far smaller than the big-M a generic gate would use.

In the integer feasible set of *this* model the two rows are in fact implied: every weight is strictly positive, so at $Y_{gkt} = 0$ the payload rows (42)–(43) already force the truck's whole load to zero. They are nevertheless written, for two reasons. (40) is strictly tighter than that implication in the relaxation, because what bounds the content of a single slot is the crush cap and not the truck payload. And the implication is a property of the truck-indexed statement and of nothing more general: the moment a layout is shared by several trucks — the restriction an implementation is tempted into, and the one solved in Section 3 — the capacity rows are written on the shared layout while the payload row is written on the truck, so load can be charged to a layout that no truck selected and (1)–(2) count it as delivered. Gating the load on the dispatch is what rules that out, and it is the form in which an implementation has to carry it.

The payload is a property of the truck:

$$\sum_{n \in N} w_n\, \overline{V}_{knt} \ \le\ pK^{w}\, Y_{pkt} \qquad \forall k \in K,\ t \in T \tag{42}$$

$$\sum_{r \in R} w^{\mathrm{f}}_{r}\, V^{\mathrm{e}}_{krt} \ \le\ pK^{w}\, Y_{skt} \qquad \forall k \in K,\ t \in T \tag{43}$$

and the crush limit a property of each stack. On plant-bound trucks it is period-exact, evaluated on the product mix actually loaded in the slot; on supplier-bound trucks it is deterministic in the layout, since the weight of a folded RTI depends only on its type:

$$\sum_{n \in N} w_n\, V_{kint} \ \le\ W^{\max} \qquad \forall k \in K,\ i \in I,\ t \in T \tag{44}$$

$$\sum_{h \in H} \sum_{r \in R(h)} w^{\mathrm{f}}_{r}\, n_{hr}\, L_{s,i,h,k,t,r} \ \le\ W^{\max} \qquad \forall i \in I,\ k \in K,\ t \in T \tag{45}$$

1.1 Symmetry breaking, slot-set sizing and data assumptions

The slots $i \in I$ of a given truck are interchangeable, so any solution admits $|I|!$ permutations of the slot index. We break this symmetry by enforcing that slot $i+1$ is used only if slot i is used:

$$\widehat{X}_{g,i+1,k,t} \ \le\ \widehat{X}_{gikt} \qquad \forall g \in G,\ k \in K,\ t \in T,\ i \in \{1, \ldots, |I| - 1\}. \tag{46}$$

The trucks of a given direction and period are interchangeable in the same way: nothing in the data distinguishes one element of K from another. We likewise force them to be used in index order,

$$Y_{g,k+1,t} \ \le\ Y_{gkt} \qquad \forall g \in G,\ t \in T,\ k \in \{1, \ldots, |K| - 1\}. \tag{47}$$

The row is valid because any solution can be re-indexed, but it removes only the choice of *which* subset of trucks is used. The loads and the layouts remain interchangeable among the used trucks, so a plan dispatching m trucks in a period still appears $m!$ times in the search, which is the difficulty Section 2 removes by aggregation rather than by a row.

Three properties of the data are assumed throughout. The cardinality of I is a modelling parameter, and an upper bound that eliminates provably unused slot variables without cutting any feasible layout is

$$|I| \;\le\; \left\lfloor \frac{pK \cdot qK}{\min_{h \in H} p_h \cdot q_h} \right\rfloor, \tag{48}$$

corresponding to tiling the truck floor with the smallest-footprint stack type; it serves as a tight preprocessing limit on $|I|$. The product-to-RTI map is a partition — each product travels in exactly one RTI type, so ρ is well defined and the families $M(r)$ are disjoint — and every RTI type carries at least one product, so no $M(r)$ is empty. Finally $lt \ge 1$: a shipment never arrives in the period it leaves, which is what the lagged terms of (9) and (16) require.

Domains

$$\begin{aligned}
&Z_{gkt},\, Y_{gkt},\, \theta_{gkt},\, X_{gihkt},\, LX_{gihkt},\, LY_{gihkt},\, A_{gijkt},\, B_{gijkt},\, C_{gijkt},\, D_{gijkt} \in \{0,1\},\\
&L_{gihktr},\, V_{kint},\, V^{\mathrm{e}}_{krt},\, I_{gnt},\, Ie_{gtr} \in \mathbb{Z}_{\ge 0},\\
&PX_{gikt},\, PY_{gikt} \ge 0.
\end{aligned}$$

Three decisions, and the alternatives they reject. Three of the rows above admit a tempting alternative that is wrong, and each is argued where it occurs rather than left to the reader. (i) The utilisation floor is imposed on the load carried (31)–(34) and not on the layer capacity designed (35)–(36); the capacity form is a consequence and is kept as a tightening, never as the requirement. (ii) The family weight ω_{pr} in that tightening is the family *maximum*: the minimum gives a strictly stronger row that is not a necessary condition of a full truck, cuts off feasible loads, and reports an optimum above the true one. (iii) The configuration binary Z_{gkt} and the dispatch binary Y_{gkt} are both kept here, because the two differ — at $Z = 1, Y = 0$ a truck carries a layout and no freight — but the difference is one the reformulation of Section 2 does not need, and Section 2.6 deletes Y from the pricing problem on an argument stated there.

2 Dantzig–Wolfe reformulation and branch and price

Two structural difficulties stand between the 3D-SCPTOP model of Section 1 and an industrial instance. The first is that the model carries a complete two-dimensional packing problem for every truck of every direction and period — $2|K||T|$ replicas, each contributing $4\binom{|I|}{2}$ big-M disjunctions (24)–(27) whose linear relaxation is weak. The second is that those replicas are interchangeable: nothing in the data distinguishes one element of K from another, so any two used trucks of the same direction and period can swap their entire layouts and loads, and every solution appears in the search a factorial number of times.

The second difficulty is the one that scales worst, because it does not depend on the loading geometry at all. It is present in the volumetric benchmark of Appendix **??** exactly as in 3D-SCPTOP, and Section 3 reports what it costs there. Operational procurement planning lives at horizons of weeks over dozens of trucks, so that is the regime the method has to serve, and it is the regime in which a decomposition earns its place.

The two difficulties invite one bad answer, and it is worth naming because an implementation of the compact model is forced into it. Rather than let every truck design its own layout, share a catalogue of n_{LC} layouts among the trucks of a direction and let each dispatched truck pick one. That collapses $2|K||T|$ packing replicas to $2n_{\mathrm{LC}}$, but it is a *restriction* of the model of Section 1 — the trucks of a direction must now agree on n_{LC} designs — so its optimum is at or above the true one and its dual bound bounds the restriction and not the problem. It also destroys the very structure the next paragraph uses, because a shared catalogue is a global resource every truck draws on. The compact 3D runs of Section 3 are of that restricted model at $n_{\mathrm{LC}} = 3$, and are labelled as such.

The reformulation developed below removes both difficulties at once, and it removes the first without restricting anything. The master couples to the geometry through nothing but the definition of the two aggregate flows (1)–(2) and one count of trucks per direction and period; every slot, orientation, non-overlap, stack-height, crush and full-truckload constraint lives inside a pricing problem. There is no index of a loading configuration anywhere in what follows, and no variable indexed by an individual truck.

2.1 The decomposition

The reformulation is derived structurally rather than by analogy, and the derivation is given first because every property claimed later — that the master is equivalent to 3D-SCPTOP, that λ may be taken integer, that a branching row leaves the pricing problem intact — is a consequence of one of its steps.

Blocks. Take the block index to be $\mathcal{B} = G \times K \times T$ and inspect Section 1. Every row from (21) to (47) — slot activation, orientation, stack height, non-overlap, cover, containment, the full-truckload disjunction in both its forms, the dispatch row, the slot and empty capacities, the dispatch gates, the payload and the crush caps, the slot symmetry — carries a single direction, a single truck and a single period as a fixed $\forall$ index or as a literal. Not one of them couples two values of g, of k or of t. The exception is (47), which relates truck $k+1$ to truck k; it is dealt with below.

The remaining rows are the objectives (6)–(7), the inventory balances (9)–(20) and the definitions (1)–(2) of the two aggregate flows. They see the blocks *only* through those two aggregates and through the count of dispatches $\sum_k Y_{gkt}$ that z^K charges for. The model is therefore block-angular over $\mathcal{B}$ with no further argument, and a block whose rows never couple two values of an index is the product of its parts over that index. This is a property of the formulation and not a discovery about it: Section 1 is written over trucks precisely so that it holds.

Delete the quantifier. The pricing problem is now obtained by deletion and by nothing else. Fix a class (g, t), take the rows listed above, and *remove the* $\forall$ *over* $\mathcal{B}$: drop the indices g, k and t from every variable, read g as data, and what is left is a mixed integer programme over one truck. That programme is (PP^g) of Section 2.6; Table 2 is the correspondence, row by row. Nothing is invented for the subproblem and nothing of the geometry is left behind in the master. The only rows that do not make the journey are the two the aggregation disposes of — (47), which has no counterpart once the blocks are counted rather than indexed, and (37), whose two binaries collapse into one (Section 2.6).

Table 2: The block rows of Section 1 and the rows of (PP^g) they become when the $\forall$ over $\mathcal{B} = G \times K \times T$ is deleted. The two directions share the geometry, so the paper writes it once with g as data; the direction-specific rows are those of Section 2.6.

Row	Section 1	(PP^g)
Block rows: the pricing problem		
Slot active	(21)	(62)
Orientation	(22)	(63)
Stack height	(23)	(64)
Non-overlap	(24)–(27)	(65)–(68)
Cover	(28)	(69)
Containment	(29)–(30)	(70)–(71)
Full truckload, volume	(31), (33)	(73)
Full truckload, payload	(32), (34)	(74)
The same on capacity	(35)–(36)	(75)
Slot capacity	(38)	(76)
Folded capacity	(39)	(80)
Dispatch gate, full	(40)	(77)
Dispatch gate, folded	(41)	(81)
Payload, plant-bound	(42)	(78)
Payload, supplier-bound	(43)	(82)
Crush limit, plant-bound	(44)	(79)
Crush limit, supplier-bound	(45)	(83)
Slot symmetry	(46)	(72)
Transport cost	(6)	$cK \cdot Z$ in (61)
Dispatch	(37)	deleted; Z carries the truck
Truck symmetry	(47)	dropped in the aggregation
Coupling rows: the master		
Aggregate flows	(1)–(2)	(56)–(57)
Fleet per class	$\lvert K\rvert$ trucks per (g, t)	(58)
Inventory and coverage	(9)–(20)	verbatim
Holding cost	(7)	second term of (55)

Identical blocks. Nothing in the data distinguishes one element of K from another: K is anonymous. The blocks of a fixed (g, t) are therefore identical, and identical blocks are aggregated. In place of one copy of the block per truck, the reformulation carries one non-negative integer variable per block *solution*, counting how many of the class's blocks take it, and one row per class bounding that count by $|K|$. This is where the second difficulty disappears: the trucks are never distinguished in the first place, so there is no permutation of them to search.

Classes. The classes are the pairs $(g, t) \in G \times T$. Within a class the block rows are pinned to the class's values of g, so there is one pricing problem per *direction*. The pricing rows read no datum indexed by t: the trailer, the stack types, the crush limit and the utilisation floor are all time-invariant. The pricing problems of the periods of one direction therefore differ *only in their duals*, and one column set serves every period of that direction. This is what makes a column priced at one period usable at another, and it is the reason the whole horizon costs

two pricing problems rather than $2|T|$.

The dropped row. The truck-symmetry row (47) of the compact model orders the trucks of a direction and period so that truck $k+1$ is used only if truck k is. It is a statement about *which* identical block carries which load, and after aggregation there is no such statement to make: counts of identical blocks carry no order, so the master has no variable in which to express it and the row is dropped.

Dropping a *valid* row is not automatically free — it could in principle be the row that made the compact model's optimum match the reformulation's — so the drop is discharged by proof rather than by appeal to the blocks being identical. The converse direction of Proposition 1 takes a master solution and assigns trucks to columns *in index order*; the resulting compact solution satisfies (47). The equivalence therefore holds between the master and the compact model *with* the row in force, which is what has to be true for the row's absence to cost nothing.

Discretisation. Every block variable that reaches the master is integer. For the plant-bound block these are the per-slot product loads V_{in}, whose sum over slots is the block's image; for the supplier-bound block they are the folded counts V^{e}_r. A block solution may therefore be taken to be an *integer* point of its pricing problem, completed by a minimum-cost choice of the block's continuous variables — the placement coordinates PX_i, PY_i, which cost nothing. Discretisation then applies: the multipliers λ are integer, not merely non-negative, and a fractional λ in the relaxation is a genuine integrality violation rather than an artefact of the reformulation.

The empty column. The point with every block variable at zero is feasible for both pricing problems, has a zero image and costs nothing. It is the slack of the count rows, which may therefore be written as inequalities and need no explicit slack variable.

2.2 Columns

A column of a direction is the image of a feasible point of that direction's pricing problem, together with the point itself as a witness. Write Ω^p for the set of plant-bound columns and Ω^s for the supplier-bound set, and index a column by ω rather than by p, which already denotes the plant direction and, as p_h, the length of a footprint.

A *plant-bound* column $\omega \in \Omega^p$ is a vector $b_\omega = (b_{\omega n})_{n\in N}$ of full RTIs — the value of the aggregate $\overline{V}_n$ at the point — for which there exist a slot count $m \le |I|$, stack types $h_1, \dots, h_m \in H$, orientations, bottom-left coordinates (x_i, y_i), layer counts ℓ_{ir} and per-slot product loads v_{in} such that

$$\text{the } m \text{ footprints, rotated as chosen, do not overlap and lie inside } pK \times qK, \tag{49}$$

$$\textstyle\sum_{r\in R(h_i)} r_{pr}\ell_{ir} \;\le\; rK \qquad \forall i \le m, \tag{50}$$

$$\textstyle\sum_{n\in M(r)} v_{in} \;=\; n_{h_i r}\ell_{ir} \qquad \forall i \le m,\ r \in R(h_i), \tag{51}$$

$$\textstyle\sum_{n\in N} w_n v_{in} \;\le\; W^{\max} \qquad \forall i \le m, \tag{52}$$

$$\textstyle\sum_{n\in N} w_n b_{\omega n} \;\le\; pK^w, \qquad \max\left\{\frac{\sum_n \gamma_{p\rho(n)} b_{\omega n}}{\Gamma_K},\ \frac{\sum_n w_n b_{\omega n}}{pK^w}\right\} \;\ge\; \alpha, \tag{53}$$

$$b_{\omega n} = \textstyle\sum_{i\le m} v_{in} \qquad \forall n \in N. \tag{54}$$

Condition (51) is written with equality where (38) has an inequality, and the two describe the same set of images: a layer that carries nothing may always be removed, which leaves the

placement unchanged, the stack no taller and no heavier, and the load — which is what (53) is a condition on — exactly as it was. The step uses the floor being stated on the load; under the capacity form an empty layer counts as occupied volume and the two readings come apart.

A *supplier-bound* column is the analogous object $e_\omega = (e_{\omega r})_{r \in R}$ over folded heights r_{sr}, folded weights w^{f}_r and folded volumes γ_{sr}; there no product assignment is needed, because the weight of a folded RTI depends only on its type. Every non-empty column costs $c_\omega = cK$, the value of $cK \cdot Z$ at the point; the empty column costs nothing.

Three features of this definition carry the method.

First, the column carries the *per-slot* assignment v_{in}, so the crush limit (52) is evaluated on the product mix actually riding in that slot. A column therefore reproduces the period-exact cap (44) of the compact model, and not the conservative surrogate $\bar{w}_r = \max_{n \in M(r)} w_n$ that a pool of capacity-only configurations is forced to use.

Second, (53) is the compact model's full-truckload disjunction (31)–(34), written on the load a truck actually carries; the capacity pair (35)–(36), written on the layers the truck's stacks provide, is the weaker necessary condition and is implied by it. A truck is full when it is full on the *binding* resource — α of volume *or* α of payload, never the sum of the two ratios, which is strictly weaker and would accept a trailer 35% full by volume and 20% by weight. Both regimes occur: coarse containers fill the trailer by volume, dense ones by weight.

Third, a column is a feasibility certificate. The placement $(h_i, x_i, y_i, \text{rotation}, \ell_{ir}, v_{in})$ witnessing $\omega \in \Omega^p$ is retained with the column, so any plan the master assembles can be handed to a loading crew as a drawing and re-verified independently of the solver that produced it. Section 3 does exactly that, truck by truck, on every reported plan.

2.3 The master problem

Let $\lambda^g_{\omega t} \in \mathbb{Z}_{\geq 0}$ be the number of trucks of class (g, t) dispatched carrying column ω, and let $\overline{V}_{nt}$ and $\overline{V^{\mathrm{e}}}_{rt}$ be the aggregate flows the inventory block reads. These aggregates are the compact model's Q_{nt} and Q_{rt} under a name that says where they come from: (56)–(57) below are the definitions (1)–(2) with each truck's load replaced by the image of the column it carries. The master problem (MP) is

$$\min \sum_{g \in G} \sum_{t \in T} \sum_{\omega \in \Omega^g} c_\omega \, \lambda^g_{\omega t} \; + \; \lambda^I \sum_{g \in G} \sum_{n \in N} \sum_{t \in T} ic_{gn} I_{gnt} \tag{55}$$

subject to

$$\overline{V}_{nt} = \sum_{\omega \in \Omega^p} b_{\omega n} \, \lambda^p_{\omega t} \qquad \forall n \in N,\ t \in T \tag{56}$$

$$\overline{V^{\mathrm{e}}}_{rt} = \sum_{\omega \in \Omega^s} e_{\omega r} \, \lambda^s_{\omega t} \qquad \forall r \in R,\ t \in T \tag{57}$$

$$\sum_{\omega \in \Omega^g} \lambda^g_{\omega t} \leq |K| \qquad \forall g \in G,\ t \in T \tag{58}$$

$$(9)\text{–}(20) \qquad \text{verbatim}$$

$$\lambda^g_{\omega t} \in \mathbb{Z}_{\geq 0}, \quad I_{gnt} \in \mathbb{Z}_{\geq 0}, \quad Ie_{gtr} \in \mathbb{Z}_{\geq 0}, \quad \overline{V}_{nt}, \overline{V^{\mathrm{e}}}_{rt} \text{ free.}$$

The aggregates are written free because no row bounds them below; (56)–(57) equate each to a non-negative combination of non-negative λ, so $\overline{V} \geq 0$ is implied and an implementation may declare it either way. That is the whole master. Objective (55) is (8) with the transport term rewritten over columns: $c_\omega = \lambda^K cK$ for every non-empty column and $c_\omega = 0$ for the empty

one, so that $\sum_\omega c_\omega \lambda^g_{\omega t}$ is $\lambda^K z^K$ and the second term is $\lambda^I z^I$. Since every non-empty column costs the same, the first term is cK times the number of trucks dispatched, at $\lambda^K = 1$. The superscripted weights λ^K, λ^I of (8) and the multipliers $\lambda^g_{\omega t}$ are different objects; both names are the ones the model and the decomposition carry in the literature, and they never appear in the same expression. Writing c_ω rather than cK is what makes the empty column free, and hence usable as the slack of the count row. Constraints (56)–(57) *define* the two aggregates as what the dispatched trucks carry; (58) caps the fleet available per direction and period, with the empty column as its slack; and the inventory balances and coverage-stock rows of Section 1 are carried over word for word, for both nodes and both kinds of stock, full and folded (Section 2.11).

Two things are worth saying about what is *not* there. There is no loading-configuration index, and so none of the model's geometric blocks: the master does not know what a slot is. And there is no variable indexed by an individual truck, so the symmetry that (47) was introduced to suppress has no representation to be suppressed in. The entire coupling between the transport–inventory problem and the loading problem is three families of rows: two definitions and one count.

Proposition 1 (equivalence)**.** *MP and 3D-SCPTOP have the same optimal value.*

Proof. Given a 3D-SCPTOP solution, each used truck (g, k, t) induces a load vector satisfying (49)–(54), obtained by reading the slot data off its own layout and setting $v_{in} = V_{kint}$; counting trucks by load vector gives a feasible MP solution, of equal cost because transport cost depends only on the number of used trucks and the inventory rows see only the aggregates, which (56)–(57) reproduce. Conversely, give each truck counted by an MP solution the layout witnessing the column it carries, assigning trucks to columns arbitrarily — the blocks being identical, any assignment serves — which reconstructs a 3D-SCPTOP solution of the same cost, feasible because each column carries a witnessing placement. No catalogue has to be sized for this: in Section 1 a truck's layout is its own. Assigning them in index order makes that solution satisfy the truck-symmetry row (47) as well, so the equivalence holds whether or not the compact model carries it. □

The proposition holds unconditionally, and that is a consequence of how Section 1 is written rather than a strength of the decomposition: there is no catalogue of layouts to be sized, so there is no hypothesis about its size to carry. It is what distinguishes this approach from a pool-based method such as the company procedure PLC-SCPTOP of Section **??**. A predefined pool is a *restriction* of the problem and can only yield upper bounds of unknown quality, whereas MP is the same problem written differently, so any bound derived from it is a bound on the original. The same qualification applies to any implementation of the compact model that shares a catalogue of n_{LC} layouts among the trucks of a direction: Section 3 compares against one run at $n_{\mathrm{LC}} = 3$, which is a restriction of Section 1, so its *plan* is a legitimate comparator and its dual bound bounds that restriction and not $z^\star$.

2.4 The restricted master problem and the reduced cost

$|\Omega^g|$ is astronomical, so MP is solved over a subset. The restricted master problem (RMP) is MP over $\bar{\Omega}^g \subseteq \Omega^g$ with integrality relaxed to $\lambda^g_{\omega t} \geq 0$; solving it yields duals, the pricing problems use those duals to look for columns of negative reduced cost outside $\bar{\Omega}^g$, the found columns are added, and the RMP is re-solved. If no such column exists the relaxation of MP has been solved over the whole of Ω^g.

Let π_{nt} and π_{rt} be the duals of the definition rows (56)–(57) and $\mu_{gt} \leq 0$ the dual of the count row (58). The reduced cost of a plant-bound column ω dispatched in period t is

$$\bar{c}(\omega, t) \;=\; c_\omega - \sum_{n \in N} \pi_{nt}\, b_{\omega n} - \mu_{pt}, \tag{59}$$

and analogously with $e_{\omega r}$ and π_{rt} in the supplier direction. The pricing problem of class (g, t) minimises $c(x) - \sum \pi \cdot \mathrm{image}(x)$ over the feasible points of the direction's block, so its optimal value $z^\star_{gt}$ gives $\min_\omega \bar{c}(\omega, t) = z^\star_{gt} - \mu_{gt}$, and a column enters when this is negative. In a Phase-I pass $c \equiv 0$ and the same problem minimises the image alone.

The sign convention is worth stating, because it is where a reimplementation goes wrong silently. With the definition row written as $\overline{V}_{nt} - \sum_\omega b_{\omega n} \lambda^p_{\omega t} = 0$ and a solver reporting reduced costs as $c - A^\top \Pi$ (gurobipy's `Pi`), the price that enters the pricing objective is $\pi = -\Pi$ for a definition row and $\mu = +\Pi$ for a count row. The prices π are irrational in general and must be rounded before entering an integer-coefficient subproblem; they are rounded *upwards*, never to nearest, so that the rounded objective dominates the true one and the value the subproblem returns remains an over-estimate of the true maximum, as the certificate of Section 2.5 requires. Rounding to nearest makes it a slight under-estimate, silently invalidates the certificate, and with reduced costs at exactly zero returns a value just above the threshold so that convergence is never declared.

Three further properties of the RMP matter for what follows.

First, it carries *artificial* variables on the definition rows (56)–(57) and on the coverage-stock rows, penalised at a large cost in the objective, so that it is feasible for any pool and always yields duals. A solution with positive artificials is not a plan.

Second, it carries cumulative cover cuts: by period t the plant must have received cumulative demand plus coverage stock, and dividing that requirement by the most a single truck can carry bounds from below the number of trucks dispatched up to $t - lt$. These rows tighten the relaxation considerably. Their duals, like those of every row the search later adds, are folded into μ_{gt} so that (59) remains the exact reduced cost. This is possible because a branching or cutting row $\sum_\omega f(\omega)\lambda_\omega \lessgtr \beta$ is *robust* — it leaves the pricing problem structurally unchanged — exactly when f is constant within a class or affine in the column's image. Every row this paper adds to the master is of one of those two kinds, which is why no branching decision ever has to be communicated to a pricing problem.

Figure 1 places the pieces in the loop they run in.

Third, the RMP is *period-scoped*: a column priced against the duals of period t receives a dispatch variable at t alone, and only the shared initial pool is offered to every period. This changes the size of the restricted master, not the master — by the Classes step of Section 2.1 the pricing problem is the same for every period of a direction, and it is still solved per (g, t) over the whole of Ω^g — so a sweep that finds nothing at any pair still proves that the restricted LP equals the master LP.

2.5 Lower bounds

Care is needed here, because the natural candidate is not a bound at all. The RMP is a *restriction* of MP: it optimises over $\bar{\Omega}^g \subseteq \Omega^g$, so its LP value satisfies $z_{\mathrm{RMP}} \geq z_{\mathrm{MP}}$. Until column generation has converged the RMP value is an *upper* bound on the master LP, and reporting it as a lower bound is simply wrong. Four rules decide what may be reported; Figure 2 states them as a decision.

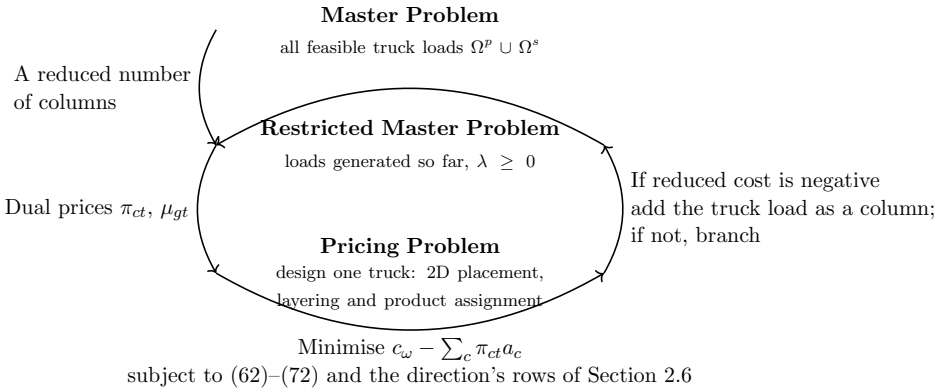


Figure 1: Branch and price scheme for 3D-SCPTOP. Every geometric, height, crush-limit, payload and full-truckload constraint is enforced inside the pricing problem, so each column is a physically loadable truck.

A certificate needs a full sweep. When the pricing problem has *proven* that $\zeta_{gt} := \max_{\omega \in \Omega^g} \sum_c \pi_{ct}\, a_{\omega c} \leq cK - \mu_{gt}$ for every g and every t, no column outside $\bar{\Omega}^g$ has negative reduced cost, so the current duals are feasible for the dual of the full MP. Weak duality then gives $z_{\text{RMP}} = z_{\text{MP}} \leq z^\star_{\text{3D-SCPTOP}}$. The proof is only as good as the exhaustiveness of the sweep behind it. Two economies that are legitimate while columns are still coming — pricing only the pairs whose aggregates are fractional, and skipping a pair whose duals round to something already priced — forfeit the right to certify, because a pair nobody looked at proves nothing. Certification therefore runs one sweep over *every* (g, t), with no memoisation and no restriction to the pairs a branch has touched, and every call must come back *proved*: an oracle stopped on its time limit reports itself unproven, and a sweep with one such pair proves nothing about the whole. Every branching fix must also be released before the sweep, since a restricted LP sits above the region's own LP and certifying with fixes in place publishes a bound that is too high by construction.

The bound is the raw LP value. The RMP objective includes the penalty on the artificial variables. The cost of a plan is the value *net* of that penalty, and that is the right number to report as a cost; the bound is the *raw* value with the penalty included, because the penalty is part of the LP whose dual the certificate rests on. Positive artificials at convergence mean the region cannot be made feasible by any column of Ω^g, so the region is infeasible in the true problem and is pruned rather than bounded.

A bound without convergence: the Lagrangian bound. A sweep that finds columns still yields a bound, as long as it was complete. With $\bar{z}$ the raw RMP LP value and $\bar{c}^\star_{gt}$ the *proved* minimum reduced cost of class (g, t),

$$\bar{z} + \sum_{g \in G} \sum_{t \in T} |K| \cdot \min\bigl(0,\ \bar{c}^\star_{gt}\bigr) \ \leq\ z_{\text{MP}}, \tag{60}$$

the multiplier being the class's own count row (58). The expression is formed only when every pair has an entry; a partial sweep yields nothing, and a pair served by a heuristic rung (Section 2.6) returns no proved minimum and so contributes none. The bound is valid at any iteration and is kept as a running maximum. Its weakness is structural, and it is why the method does not rest on it: the multiplier is the fleet size $|K|$ at each of the $2|T|$ pairs, so a reduced cost

of $-\varepsilon$ at every pair subtracts $2|T|\,|K|\,\varepsilon$ from $\bar{z}$, while the quantity the bound has to make up — the distance from the restricted LP to the true one — is typically a small fraction of one truck. Section 3.8 reports what it returns in practice.

The restricted value is never a bound. At every node the search first solves the LP over the current pool. That value sits *above* the region's true LP value and is used only as a filter: if even the over-estimate lies below the incumbent, the region cannot be pruned and the expensive sweep is not run. It is never entered as a bound, and a search stopped by its deadline reports the last *certified* value it holds, not the last LP it solved. The reason the qualification matters is that the *best* reduced cost of a class need not decrease monotonically along a column generation run: the duals move with every column added, so a pair that prices out at nearly zero under one dual vector can price out heavily under the next. "The oracle found nothing" is therefore not convergence unless every pair was proved in the same sweep.

A bound by inclusion. The certificate above is expensive when the oracle is the geometric pricing problem of Section 2.6; the following observation makes a cheaper one available.

Proposition 2 (bound by inclusion)**.** *Let $\Omega^g_{\mathrm{vol}} \supseteq \Omega^g$ be the set of loads that fit the trailer's volume and the payload and clear the full-truckload floor on the binding resource, and let z_{vol} be the LP value of the master* (55)–(58) *over Ω^g_{vol}, certified as above. Then $z_{\mathrm{vol}} \le z^\star_{\text{3D-SCPTOP}}$, and the same holds inside any region defined by bounds on the fleet vector.*

Proof. Two conventions of Section 1 are used and are stated because the inclusion rests on them: the trailer volume is $\Gamma_K = pK \cdot qK \cdot rK$, and the volume γ_{gr} charged to an RTI of type r in direction g is the volume of its bounding box, $p_h\, q_h$ times its height in that direction, with no allowance for void space inside it or clearance between stacks. Both are what the data of Section 3.2 provide. A valid placement satisfying (49)–(54) then has its RTIs in disjoint boxes inside the trailer, so their bounding-box volumes sum to at most Γ_K; its weight is at most pK^w by (53); and (53) is the same floor in both sets. Hence $\Omega^g \subseteq \Omega^g_{\mathrm{vol}}$. The rows of the master — the two definitions, the count, the inventory and coverage rows, and any bound on a dispatch total — do not depend on the load model; only the column set does. The 3D master is therefore the volumetric master with columns removed, its LP value is at least z_{vol}, and its optimum is at least its LP value. □

The practical consequence is the part that matters. A geometric pricing sweep costs $2|T|$ calls at the pricing limit and, on the instances of Section 3, certified no region on any run; the volumetric relaxation is a two-constraint knapsack oracle over the same master, and a sweep of it closes in seconds (Section 3.8). The method therefore takes its bound from the volumetric relaxation and asks geometric pricing only for columns. The division of labour is principled rather than expedient: the plan comes from the pool, the construction and the tree, the bound comes from a relaxation whose bound is valid for the problem actually being solved, and the gap between them is a genuine optimality gap for the 3D problem.

2.6 The pricing problems

By the Blocks and Classes steps of Section 2.1 the pricing problem of class (g, t) *is* the block of 3D-SCPTOP with the quantifier over $\mathcal{B}$ removed: the same rows, with the direction g supplying the data and the master supplying the duals. Table 2 is the correspondence. Nothing has to be invented for it, and nothing of the geometry is left behind in the master. There is one such problem per direction, written (PP^g); the periods of a direction share it and differ only in π

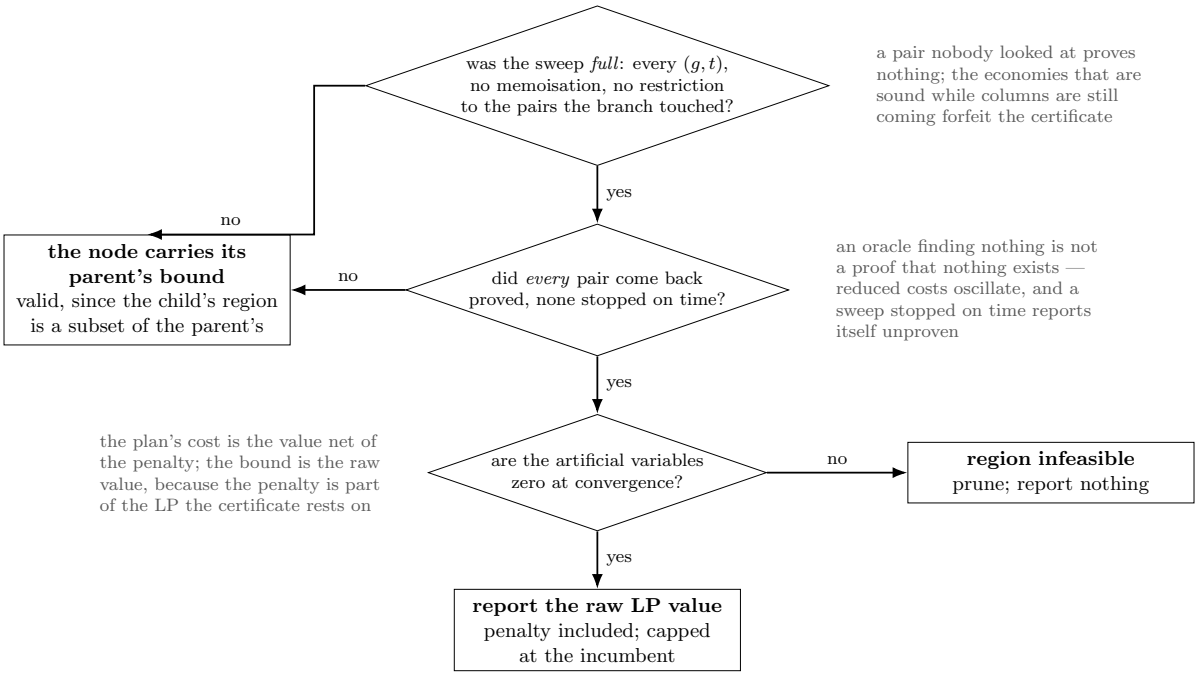


Figure 2: What may be reported as a lower bound. The restricted-master LP value is an *upper* bound on the master LP until pricing has converged, so it is used only as a pruning filter; a node obtains a bound of its own only from a full sweep in which every pair is proved, and otherwise inherits its parent's. A quantity above a solution one can exhibit is not a loose bound but a false statement, and it inflates every optimality gap computed from it, so the reported value is additionally capped at the incumbent.

and μ. One row of the block does not survive the deletion unchanged — the dispatch row (37), whose two binaries are identified below — and the reason is given with (74).

Let $\mathcal{I}$ index the slots, bounded by (48), and drop the block index from the compact model's variables. Table 3 lists them. The two directions share the whole of the geometry, which is therefore written once with g as a parameter; they differ only in the rows of Section 2.6.

Table 3: Nomenclature of the pricing problem (PP^g). The variables are those of a single block of Section 1 with the block index dropped; g and the duals π, μ are data.

Decision variables	
Z	Binary. A truck is dispatched under this column — equivalently, the block designs a layout and sends it — and $c(x) = cK \cdot Z$; at $Z = 0$ the point is the empty column
X_{ih}	Binary. Slot i holds a stack of type h; $\widehat{X}_i := \sum_h X_{ih}$ is slot i used
LX_{ih}, LY_{ih}	Binary. The stack in slot i is oriented with p_h along x, respectively along y
PX_i, PY_i	Bottom-left coordinates of slot i on the trailer floor
L_{ihr}	Integer. Layers of RTI type r in slot i under stack type h; $\overline{L}_{hr} := \sum_i L_{ihr}$
$A_{ij}, B_{ij}, C_{ij}, D_{ij}$	Binary. Which of the four separating half-planes resolves the overlap of slots i and j
θ	Binary. The full-truckload requirement is met on volume ($\theta = 1$) rather than on payload ($\theta = 0$)

V_{in}	Integer, $g = \text{p}$ only. RTIs of product n in slot i; $\overline{V}_n := \sum_i V_{in}$ is the column's image
V_r^{e}	Integer, $g = \text{s}$ only. Folded RTIs of type r in the truck; the column's image

The shared block: geometry

$$\min \; cK \cdot Z \; - \; \sum_c \pi_{ct}\, \text{image}_c \qquad \bar{c} = z - \mu_{gt} \tag{61}$$

$$\widehat{X}_i \le Z \qquad \forall i \in \mathcal{I} \tag{62}$$

$$LX_{ih} + LY_{ih} = X_{ih} \qquad \forall h \in H,\, i \in \mathcal{I} \tag{63}$$

$$\sum_{r \in R(h)} r_{gr}\, L_{ihr} \le rK \cdot X_{ih} \qquad \forall h \in H,\, i \in \mathcal{I} \tag{64}$$

$$PX_i + \sum_h (p_h LX_{ih} + q_h LY_{ih}) \le PX_j + M^x(1 - A_{ij}) \qquad \forall i < j \tag{65}$$

$$PX_j + \sum_h (p_h LX_{jh} + q_h LY_{jh}) \le PX_i + M^x(1 - B_{ij}) \qquad \forall i < j \tag{66}$$

$$PY_i + \sum_h (q_h LX_{ih} + p_h LY_{ih}) \le PY_j + M^y(1 - C_{ij}) \qquad \forall i < j \tag{67}$$

$$PY_j + \sum_h (q_h LX_{jh} + p_h LY_{jh}) \le PY_i + M^y(1 - D_{ij}) \qquad \forall i < j \tag{68}$$

$$A_{ij} + B_{ij} + C_{ij} + D_{ij} \ge \widehat{X}_i + \widehat{X}_j - 1 \qquad \forall i < j \tag{69}$$

$$PX_i + \sum_h (p_h LX_{ih} + q_h LY_{ih}) \le pK \qquad \forall i \in \mathcal{I} \tag{70}$$

$$PY_i + \sum_h (q_h LX_{ih} + p_h LY_{ih}) \le qK \qquad \forall i \in \mathcal{I} \tag{71}$$

$$\widehat{X}_{i+1} \le \widehat{X}_i \qquad \forall i \le |\mathcal{I}| - 1 \tag{72}$$

The shared block: the full-truckload disjunction

A truck is full on the binding resource. Written on the load the truck actually carries, with Γ_K the trailer volume and α the utilisation floor,

$$\sum_c \gamma_{g\rho(c)}\, \text{image}_c \;\ge\; \alpha\, \Gamma_K\, (Z - 1 + \theta) \tag{73}$$

$$\sum_c w_c\, \text{image}_c \;\ge\; \alpha\, pK^w\, (Z - \theta). \tag{74}$$

At $Z = 1$ exactly one of the two is active and the other is vacuous, so (73)–(74) is the disjunction (53) and not the sum of the two ratios, which is strictly weaker and would accept a trailer 35% full by volume and 20% by weight.

These two rows are where the block's second binary goes, and the argument is worth stating because deleting a variable from a formulation is a claim about its feasible set. Section 1 carries two binaries per truck: Z_{gkt}, the truck is configured, and Y_{gkt}, the truck is dispatched, related by (37). With no loading-configuration index they differ at exactly one kind of point, $Z = 1$ with $Y = 0$: a truck that carries a layout and no freight. By (40)–(41) its image is 0, and by (6) its cost is 0 — which are precisely the image and the cost of the empty column $Z = Y = 0$.

The map from feasible points to columns therefore sends every such point to a column the block already has, so identifying the two binaries removes points without removing a single element of Ω^g, and the master, which sees only images and costs, cannot tell the difference. The block below keeps Z, charges it $cK \cdot Z$, and hangs every load gate on it.

It is worth doing rather than merely harmless. As two variables, $Z = 1$ with $Y = 0$ admitted a whole family of zero-image, zero-cost points with the slot, orientation and non-overlap variables still free — a symmetry that a big-M packing model solved in the inner loop can ill afford. And it says something true about the reformulation: in Dantzig–Wolfe the column *is* a truck, and how many trucks take it is what λ counts in the master, so a second "is it used" binary inside the block asks a question the master already answers. One binary is the right number. (The same observation licenses fixing $Z = 1$ when the empty column is wanted out of the way: the empty column is one analytically known point, of image 0, cost 0 and reduced cost $-\mu_{gt} \geq 0$, so it never prices in.)

Four redundant rows are available, two on the layer *capacity* rather than on the load and two on the occupied floor area:

$$\sum_{h,\, r \in R(h)} \gamma_{gr}\, n_{hr}\, \overline{L}_{hr} \;\geq\; \alpha\, \Gamma_K\, (Z - 1 + \theta), \qquad \sum_{h,\, r \in R(h)} \omega_{gr}\, n_{hr}\, \overline{L}_{hr} \;\geq\; \alpha\, pK^w\, (Z - \theta),$$

$$\sum_i \sum_h p_h q_h X_{ih} \;\leq\; pK \cdot qK, \qquad \sum_i \sum_h p_h q_h X_{ih} \;\geq\; \alpha\, pK \cdot qK\, (Z - 1 + \theta). \tag{75}$$

The first pair is (35)–(36) with the block index dropped, implied by (73) because the load never exceeds the capacity; the second says that the footprints lie on the floor and that a slot contributes at most its footprint area times rK of volume, so a truck full by volume has at least α of its floor occupied. All four are implied, and any of them may be added to help a solver find a *first* feasible layout, which on tight instances is harder than optimising once one is known. The oracles behind Section 3 carry the second pair.

The direction of that implication is the whole point, and it is worth being explicit because the reverse is tempting and wrong. Writing the floor on the stack capacity *alone*, with a per-type weight in place of the product's own, gives a strictly weaker condition: it admits a point whose stacks could have been filled to the floor but whose actual load falls below it, and such a point is not a column. A row over a concrete load must use that load's own product weights; only a row written on the layer capacity, which does not yet know which products will ride in the stack, may use the heaviest the family can carry, and then only because the product-exact condition is imposed alongside it. Getting this backwards is silent in both directions — the family maximum in a per-load floor passes trucks that are not full, and the family minimum in a capacity floor cuts off feasible ones.

The direction-specific rows

Plant-bound, $g = \mathrm{p}$, with $\mathrm{image}_n = \overline{V}_n$ and $\bar{u}^V_n$, $\bar{u}^{\mathrm{e}}_r$ the analytic slot and truck bounds of Section 1:

$$\sum_{n \in M(r)} V_{in} \;\leq\; \sum_{h\,:\, r \in R(h)} n_{hr}\, L_{ihr} \qquad \forall i \in \mathcal{I},\, r \in R \tag{76}$$

$$V_{in} \;\leq\; \bar{u}^V_n\, Z \qquad \forall i \in \mathcal{I},\, n \in N \tag{77}$$

$$\sum_{n \in N} w_n\, \overline{V}_n \;\leq\; pK^w\, Z \tag{78}$$

$$\sum_{n \in N} w_n\, V_{in} \;\leq\; W^{\max} \qquad \forall i \in \mathcal{I} \tag{79}$$

Supplier-bound, $g = \mathrm{s}$, with $\mathrm{image}_r = V^{\mathrm{e}}_r$:

$$V^{\mathrm{e}}_r \;\le\; \sum_{h\,:\,r\in R(h)} n_{hr}\,\overline{L}_{hr} \qquad \forall r \in R \tag{80}$$

$$V^{\mathrm{e}}_r \;\le\; \bar{u}^{\mathrm{e}}_r\,Z \qquad \forall r \in R \tag{81}$$

$$\sum_{r\in R} w^{\mathrm{f}}_r\,V^{\mathrm{e}}_r \;\le\; pK^{w}\,Z \tag{82}$$

$$\sum_{h}\sum_{r\in R(h)} w^{\mathrm{f}}_r\,n_{hr}\,L_{ihr} \;\le\; W^{\max} \qquad \forall i \in \mathcal{I} \tag{83}$$

The asymmetry is the one the compact model already has. A plant-bound truck needs the per-slot assignment V_{in} because the crush limit depends on which products ride in the slot (79); a supplier-bound truck does not, because the weight of a folded RTI depends only on its type, so (83) is a statement about the stack and not about its contents.

What the method requires of the oracle, and what it does not. The pricing problem is the mixed integer programme above, and that is what the method specifies: the oracle's only obligation is to return an optimal point of *this* feasible set for the duals it is handed. The method is stated over the subproblem and not over a solver for it, and nothing in the master, the bound or the search reads anything else about how the point was obtained.

The hard core of the programme is the disjunctive non-overlap relation (65)–(69): four big-M rows per pair of slots, whose linear relaxation is weak for the same reason it is weak in the compact model. An equivalent statement avoids those rows altogether. Non-overlap of the rectangles $[PX_i, PX_i + \delta^x_i) \times [PY_i, PY_i + \delta^y_i)$, with side lengths $\delta^x_i = \sum_h (p_h LX_{ih} + q_h LY_{ih})$ and $\delta^y_i = \sum_h (q_h LX_{ih} + p_h LY_{ih})$ — the expressions already appearing in (65)–(71) — can be imposed as a single two-dimensional disjointness relation over those rectangles, with no separating binaries A, B, C, D at all. The feasible sets of the two statements coincide, so an implementation is free to use either. The runs reported in Section 3 were produced by the second, through a constraint programming solver that propagates that relation natively; Section 3.1 records which solver solved what. That is an implementation choice behind those numbers and not a part of the method, and Section 3 reports no comparison between the two technologies: the one on record was made under an oracle that did not implement the composition bounds the method depends on, which makes it a comparison between two different subproblems rather than between two solvers. What matters is only that the oracle solve *this* problem; an earlier variant that imposed the full-truckload requirement as the sum of the two ratios in (53) rather than as their disjunction solved a strictly weaker one and returned a column that failed verification.

The volumetric oracle. The same master — the same two definition rows, the same count row, the same inventory balances and coverage stock — is also solved over the loads of the volumetric benchmark of Appendix **??**, where a load is a column iff its volume fits the trailer's, its weight the payload, and it clears the full-truckload floor on the binding resource. Pricing is then a two-constraint integer knapsack that closes in milliseconds. Running both oracles through one master and one search is what makes the comparison of Section 3 an experiment about the loading representation rather than about two solution technologies, and it is the setting in which per-node certification is affordable.

What a certificate costs. Figure 3 shows the oracle and what it can and cannot return. A certified node is a full sweep, $2|T|$ calls each closed to proven optimality. On the volumetric

oracle that is affordable at every node. On the geometric problem it is not — a single call on the instances of Section 3 takes tens of seconds to minutes, and a sweep is then the whole budget of a node — so with the geometric oracle the certificate is taken from the volumetric relaxation (Proposition 2) and geometric pricing is asked only for columns.

A pricing ladder. Any column of negative reduced cost improves the restricted master; the *most* negative one is needed only for the certificate. Column generation over the geometric problem therefore runs as a ladder. The first rung is a heuristic run *under the current duals* at every iteration — a randomised greedy construction over enumerated stack fills, placed by a shelf packer, together with a re-fill of the slot geometry of columns already in the pool — whose output is pricing output, never seeding. A heuristic load *qualifies* only if its reduced cost lies below a margin of half a percent of the pair's threshold; the best two qualifying loads per pair are kept, and a pair at which nothing qualifies goes to the exact oracle. The oracle itself is run in a stop-early mode that returns its first incumbent beating the threshold by the same margin.

What the margin does, and what it does not. The margin is worth stating precisely, because the obvious reading of it is wrong and we have measured the difference. It tests whether a heuristic column is good enough *in its own reduced cost* to be worth adding: it discards the near-zero columns a stalled greedy produces, which would otherwise enter the pool and pre-empt the oracle for nothing. It does *not* compare the heuristic column with the column the oracle would have returned at the same duals, and it cannot, since that column is exactly what has not been computed. A strongly negative heuristic column therefore passes the margin and suppresses the oracle at that pair completely. Three sweeps on $I_16_15_2_90_k25$ at five seconds per call record the extreme case: the rung qualified at every pair that had an improving column to give, not one priced column entered the pool, and the exact oracle — which in that experiment was run at every pair regardless — therefore contributed nothing.

That matters only because the two generators are *complementary rather than substitutable*, which is the same $\mathrm{conv}(\Omega^g)$ argument as Figure 4. In the arm that calls both at every pair, and so compares them under one dual trajectory, the oracle's columns carry 4.72 distinct commodities on average (median 5) with 0.522 of the load in the heaviest, and the rung's carry 10.17 (median 14 of 16) at 0.374: pricing returns extreme, concentrated loads because its objective is linear, and the rung returns balanced ones because it is not optimising that objective at all. So running both is not redundant work. At an equal number of sweeps from the same pool, running both gives a tighter restricted-master LP than either generator alone at either per-call budget — 76 304.92 against the pre-emptive ladder's 76 839.25 over three sweeps at five seconds, and 75 361.05 against 75 983.75 over six at twenty, the exact-only arm being behind both in each case. Lower is tighter: the restricted-master value is an upper bound on the master LP until pricing converges (Section 2.5).

What stops that from being a blanket rule is the price. Running both pays an oracle call at every rung-served pair and carries the larger pool, and the master LPs then pay for both: 265 columns against the pre-emptive ladder's 185 at the same budget. Per *second* the pre-emptive arrangement is the better one at the short end, reaching 75 983.75 in 405.6 s where the additive arrangement stood at 76 498.48 at a comparable 395.9 s and needed 799.1 s for its own final value. The rule the implementation therefore applies is a condition on the oracle's marginal cost rather than a preference between the generators: **run both while the sweep's mean per-pair oracle time is at or under five seconds, and let the rung pre-empt above it**, the mean being a quantity the sweep already records. Under the per-call budget

of the campaign, 20 s, that condition selects pre-emption — which is the structure the code already had — so *nothing in the results of Section 3 was produced by a different algorithm.* Its provenance is two per-call budgets on one instance, and the two configurations also differ in cores, so it is a calibration and not a law; what the measurements fix unambiguously is its direction.

The certifying sweep never uses either rung, and neither does the Lagrangian bound (60): a certificate is one uncapped exact sweep, exactly as in Section 2.5, so the ladder changes which columns arrive and how fast, and nothing about what may be claimed.

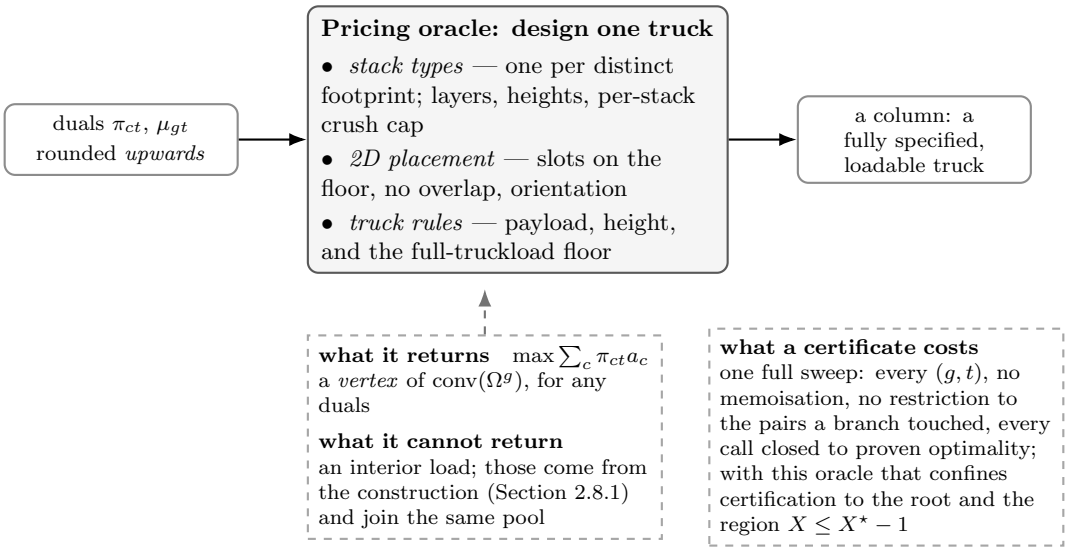


Figure 3: The pricing subproblem. Geometry, height, crush limits, payload and the full-truckload rule are enforced *inside* the oracle, so any column it returns is a physically loadable truck. It maximises the dual-weighted load and therefore returns a vertex of conv(Ω^g) whatever the duals are; loads in the interior of that hull (Figure 4) are supplied by the construction of Section 2.9, not by pricing. A certificate needs every call of a sweep closed to proven optimality, which on this oracle is the whole budget of a node, and is therefore spent only where a bound changes what the tree can conclude.

2.7 Initial columns

The search starts from the plainest possible pool: one maximally full truck per commodity and direction, obtained in the geometric case by running the pricing problem for that commodity alone — so that where it closes, the pool truck is the placeable maximum of the commodity and the same number serves as its placeability cap in Section 2.10 — and in the volumetric case by filling to capacity. The geometric probe carries a time limit like every other, and where it expires the pool still gets a column but the count behind it is not proved maximal and the analytic cap stays in force, exactly as Section 2.10 states. It is not a formality: on $I_16_15_2_90_k25$ seventeen of the eighteen (direction, commodity) probes closed and one reached its limit. No demand mixes, no diversification and no reduced variants are seeded, so whatever the search then reaches, it reached by pricing, branching and construction. This is a requirement of the method rather than a convenience: a pool assembled by hand contaminates every claim about what the search finds, and an earlier seeded pool was removed on that ground. The single-commodity pool guarantees that no direction is ever left without columns, and because these columns are shared they are offered to every period.

What such a pool, and any pool grown from it by pricing alone, *cannot* contain is shown in Figure 4 on a real pool. For any dual vector the pricing objective (61) is linear in the image, so it attains its optimum at an extreme point of $\mathrm{conv}(\Omega^g)$ and pricing returns extreme loads only. A balanced load lying inside the hull is dominated componentwise by a fractional blend of single-commodity trucks; the relaxation buys that blend for the price of one truck, while an integer plan must pay for each truck it mixes. No choice of duals returns the interior load, however the search moves them. Interior loads are precisely what an integer plan needs when a period's demand does not decompose into full trucks, and the method therefore has a second generator for them.

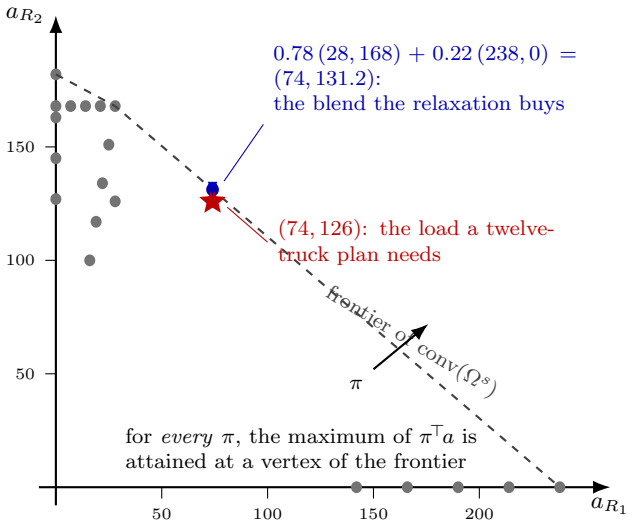


Figure 4: Why the load a cheaper plan needs is never priced. Dots are the supplier-bound loads obtained from a sweep of demand-derived price vectors; the dashed line is the upper-right frontier of their convex hull. The pricing objective is linear, so for *any* dual vector π its optimum is attained at a vertex of that frontier. The load $(74, 126)$ lies strictly below it — it is dominated componentwise by the blend $0.78\,(28, 168) + 0.22\,(238, 0) = (74, 131.2)$ — so no choice of duals returns it. The relaxation buys that blend for one truck, while an integer plan must pay for both of the trucks it mixes. The construction of Section 2.9 reaches the interior directly: it partitions a delivery vector into exact loads instead of asking a linear objective for them.

2.8 Relax and fix

The second generator, and the procedure that produced every plan reported in Section 3, is a relax-and-fix heuristic on the restricted master. Two such procedures appear in this work and they are not variants of one another; they relax and fix different things, they are run in different places, and each is stated in full below so that neither is mistaken for the other.

- The **flow-first** relax and fix of Section 2.8.1 fixes the *aggregates* — the delivery vectors and the dispatch totals — while leaving the column variables continuous, and then realises the fixed aggregates as trucks. It runs at every node of the fleet tree of Section 2.9 and is the primal heuristic of MATH-3D-SCPTOP.

- The **horizon-walk** relax and fix of Section 2.8.2 fixes the *column variables of one period at a time*, with a sliding window of integer periods and a continuous tail. It was used in

the earlier period-scope experiments on the volumetric master and is reported as such; no 3D result in this paper comes from it.

2.8.1 Flow-first relax and fix

The procedure decides first how much moves and how many trucks carry it, and builds the trucks afterwards. It is run inside a region $\mathcal{R}$ of the fleet tree — a set of bounds on the dispatch totals $n_{gt} := \sum_\omega \lambda^g_{\omega t}$ — and must stay inside it, or the children the node induces would partition around a point the node excludes. It runs on a *fresh copy* of the restricted master, because step 2 changes variable types and doing that to the node's own LP would leave it a mixed integer programme with no duals for whatever runs next.

1. *Restricted master.* The LP over the pool, inside $\mathcal{R}$, gives the duals and the fractional point the node stands on.

2. *Aggregate integer master.* Introduce the dispatch totals n_{gt} as explicit integer variables, make the aggregates $\overline{V}_{nt}$ and $\overline{V^{\mathrm{e}}}_{rt}$ integer, and leave the column variables $\lambda^g_{\omega t}$ *continuous*; impose $\mathcal{R}$ on the n_{gt} as well as on the columns. Under 3D loading add the stack-count rows of Section 2.10. Solve to a relative gap ε_2, with the solver set to find feasible solutions early.

   This is where the objective is actually decided, and it is decided *without a single call to a pricing problem*: the row set is the master's own, and what it chooses is the flows and the fleet. Three outcomes are distinguished. No solution within the limit abandons the node. Artificial slack above a tolerance means the region is infeasible over this pool, so its delivery vectors are meaningless and the node is abandoned — partitioning them would waste the node and could return a "plan" that does not meet demand. Slack below the tolerance means the integer solve ran out of time short of a clean solution; the partitions built from it are still legal loads, and step 4 decides. Write $Q^\star$ for the integer delivery vectors and $n^\star$ for the integer fleet it returns.

3. *Partition, as a cover.* For each direction and period, realise $Q^\star_{gt}$ as $n^\star_{gt}$ legal trucks by a small integer programme over per-truck commodity counts a_{kc}, $k \le n^\star_{gt}$:

$$\min \quad \sum_c \gamma_{gc}\Big(\sum_k a_{kc} - Q^\star_{gtc}\Big) \tag{84}$$

$$\text{s.t.} \quad \sum_k a_{kc} \;\ge\; Q^\star_{gtc} \qquad \forall c \tag{85}$$

$$\textstyle\sum_c \gamma_{gc} a_{kc} \le \Gamma_K, \qquad \sum_c w_c a_{kc} \le pK^w \qquad \forall k \tag{86}$$

$$\textstyle\sum_c \gamma_{gc} a_{kc} \ge \alpha\Gamma_K z_k, \qquad \sum_c w_c a_{kc} \ge \alpha pK^w(1 - z_k) \qquad \forall k \tag{87}$$

$$\textstyle\sum_c w_c a_{kc} \;\ge\; \sum_c w_c a_{k+1,c} \qquad \forall k < n^\star_{gt} \tag{88}$$

$$a_{kc} \in \{0, \dots, \bar{u}_{gc}\}, \qquad z_k \in \{0, 1\}.$$

   Constraint (85) is a *cover*, not an exact split: the parts must sum to *at least* the delivery vector in every commodity, and (84) minimises the excess. (86)–(87) make each part a legal truck, with z_k selecting the resource on which part k meets the floor; (88) orders the parts by weight to break the symmetry among them, and $\bar{u}_{gc}$ is the placeability cap of Section 2.10. If $n^\star_{gt}$ parts cannot be formed, $n^\star_{gt}+1$ and then $n^\star_{gt}+2$ are tried, provided $\mathcal{R}$ allows it. A timeout is not a proof: only a model that comes back infeasible says the cover is impossible.

4. *Integer master over pool plus parts.* The parts are added to the pool as columns and the integer master is solved over the enlarged pool, warm-started with the partition itself. Its solution is the node's plan and its value the candidate incumbent.

Why the partition must be a cover. This is the step at which the procedure could quietly become wrong, and the reason is worth stating exactly. In the compact model and in the integer master the packing is a decision variable, so $\overline{V}_{nt} = \sum_{\omega} b_{\omega n} \lambda^{p}_{\omega t}$ with integer λ is a sum of placeable loads by construction and an unrealisable delivery vector can never be requested. Step 2 breaks that link: it fixes the aggregates over *continuous* λ, so $Q^{\star}$ is a fractional blend of legal loads — and a fractional blend of legal loads need not be realisable as legal loads at all, for *any* number of trucks.

The mechanism is that floor area is consumed in whole footprints while the utilisation floor is measured in volume. A partial stack occupies the same footprint as a full one, so splitting a delivery vector into more trucks consumes more floor without adding volume, while splitting it into fewer violates the trailer. On $I_16_15_2_90_k25$ the vector $\{R_1 : 200,\ R_2 : 30\}$ holds 1.83 full-truckload floors of volume — so one truck cannot carry it and two cannot both be full — and needs $35.5\,\mathrm{m}^2$ of stacks on a $32.64\,\mathrm{m}^2$ floor, so it is realisable by no number of full trucks whatever. Under a cover the same vector is two trucks of $\{R_1 : 100,\ R_2 : 15\}$ plus a few units. An exact split would therefore fail on vectors the model has every right to choose, and failing is not a neutral outcome: it costs the node its plan.

The cover is sound for the same reason the step exists. Every part is still a legal, full truck and hence a legal column, and the parts enter step 4 *as columns with a warm start*, not as fixed dispatches, so the integer master is free to dispatch fewer of them than were built. A plan that over-delivers is feasible in the master: the inventory rows are balances with no upper bound on the plant's stock, so the surplus is priced as holding cost, and the supplier's own stock balance stops a column that would ship more than the supplier has from being dispatched at all. The excess in (84) is measured in volume rather than in units, because volume is the resource the floor and the trailer are measured in, so a surplus folded empty and a surplus full pallet are weighed by what they displace rather than counted alike.

What step 2 must know, and what it need not. Step 2 sees the master's rows and the pool, not the geometry, so it can choose aggregates that no partition can realise. The aggregate stack-count rows of Section 2.10 are what keep that in check, and the failure they fix is concrete: on $I_8_5_2_90$ over the pure pool, step 2 under a *linear* stack-area row chose the right flow — the twelve-truck optimum — and then asked the partition for $\{R_1 : 85,\ R_2 : 117\}$ in one supplier truck, which is $13 + 17 = 30$ stacks of seven layers, $32.88\,\mathrm{m}^2$ on a $32.64\,\mathrm{m}^2$ floor. The linear row admits it ($11.66 + 20.06 = 31.7\,\mathrm{m}^2$); the partition refused it; the pair went to two trucks and the plan to thirteen at 19 682.36 instead of twelve at 18 796.96. With integer stack counts per footprint step 2 sees the difference. The rows remain a relaxation of the per-truck split — an aggregate that fits n floors need not divide into n — which is why the $n + 1$ fallback of step 3 stays in the procedure by design.

2.8.2 Horizon-walk relax and fix

The second procedure attacks a different obstruction: the integer master over the whole horizon. At twenty periods that monolith is what fails. In the eight recorded runs reported below, over final pools of between 551 and 3 518 columns — the range over all eight runs — the closing integer master over the whole horizon produced no incumbent on *any* run, while the walk produced one on six of the eight.

So commit period by period. Let w be a window width. At step i of the walk the dispatch variables $\lambda^g_{\omega t}$ are *integer* for t in the window $\{t_i, \ldots, t_{i+w-1}\}$ and *continuous* for every other period, which leaves the tail free to absorb whatever the committed prefix leaves. The window's own mixed integer programme has $|\bar{\Omega}| \times w$ integer variables instead of $|\bar{\Omega}| \times |T|$. Solve it, fix the variables of its *first* period at the counts it chose, slide the window forward by one, and repeat. The walk runs on a copy of the master, for the same reason step 2 above does.

Inventory is what couples the periods, and it is forgiving in one direction: shipping earlier only carries stock, so a committed prefix rarely strands the tail. That is why the walk is cheap where the monolith is intractable. When a window does come back with positive artificials, the committed prefix cannot be completed out of the columns that window may use — under period scoping, the loads priced at the window's own periods — and two recoveries are available. A *repair* asks the pricing problem for a load covering the shortfall the artificials name and retries the same window; a *backtrack* releases the last commitment, widens the window by one and lets the two periods be decided together rather than one after the other. If neither applies the walk abandons, and records which period and which shortfall, since a walk that returns nothing silently is indistinguishable from one that was never run.

This is the procedure the period-scope experiments used, on the volumetric master, and Section 3.4 reports what it returned there. It is described here, at the length its reimplementation needs, because it is a distinct procedure with recorded results of its own, and because the aggregate-first alternative above is a response to the same obstruction — not because the paper compares the two. They have never been run against each other under a common budget, and no such comparison is offered.

2.9 The fleet tree

What may be branched on is decided before anything else, and it is decided by the criterion of Section 2.4: a row $\sum_\omega f(\omega)\, \lambda^g_{\omega t} \lessgtr \beta$ added to the master is *robust* — it changes the duals and nothing else, leaving the pricing problem structurally untouched — exactly when f is constant within a class or affine in the column's image. The branching hierarchy used here is read off that criterion rather than chosen and then justified, and it has three rungs.

1. $f \equiv 1$**: counts of dispatches.** Any bound on a sum of $\lambda^g_{\omega t}$ with unit coefficients is robust, because its dual folds into μ_{gt} exactly as the count row (58) does and the pricing objective (61) is unchanged. The aggregates available at this rung are the per-class dispatch total $\overline{\lambda}_{gt} = \sum_\omega \lambda^g_{\omega t}$, any subset sum of them, and in particular the total fleet $X = \sum_{g,t} \overline{\lambda}_{gt}$. All are integer in every plan, so a fractional one is a genuine violation to branch on.

2. f **affine in the image: deliveries.** The aggregates $\overline{V}_{nt} = \sum_\omega b_{\omega n} \lambda^p_{\omega t}$ and $\overline{V^{\mathrm{e}}}_{rt}$ have $f(\omega) = b_{\omega n}$, which is affine (indeed linear) in the image, so a bound on them is robust as well: its dual is absorbed into π_{nt}, which is a price on the image and therefore already an input of the pricing problem. These are continuous variables of the relaxation, but they count physical RTIs and are integer in any plan, so they too may carry a dichotomy.

3. **Everything else: column variables.** Bounding a single $\lambda^g_{\omega t}$ has $f = \mathbf{1}[\omega = \omega_0]$, which is neither constant within the class nor affine in the image. Such a row is *not* robust: honouring it requires excluding ω_0 from the pricing problem's feasible set, which the subproblem must be told about and which destroys the property that one column set serves every period of a direction.

The third rung is useless here as well as non-robust, which settles the matter. Bounding one $\lambda^g_{\omega t}$ removes one load from the relaxation, but a Dantzig–Wolfe master of this kind holds many columns that differ only in detail, so the relaxation answers by shifting the same fractional weight onto a near-identical column: the bound $\lambda^g_{\omega t} \le \lfloor \lambda^\star \rfloor$ excludes one point of Ω^g from a set whose remaining points include many differing from it in a single RTI, so the child's LP value moves by almost nothing and the fractionality reappears one column over. The depth needed to exhaust that family grows with the pool, which is what makes the rule unusable here rather than merely slow. Run to exhaustion on one benchmark instance with the pool frozen, it branched 3 000 times without ever producing an integral relaxation.

That leaves the first two rungs, and between them the first is the one that binds. Transport cost is $cK \cdot X$ and nothing else in the objective counts trucks, so a bound on a dispatch count is a bound on cost directly, whereas a bound on a delivery is answered by the relaxation moving freight between periods at the price of one period's holding cost — and a bound on a *single* period's dispatch total is answered the same way, by moving a truckload into a neighbouring period. The aggregate that has no such escape is the total over the whole horizon.

We therefore branch on the *fleet*. With $n_{gt} = \overline{\lambda}_{gt}$ the dispatch total of direction g in period t, let $\mathbf{n} = (n_{gt})_{g,t}$ be the fleet vector and $X := \sum_{g,t} n_{gt}$ the total fleet. Every bound placed on either is a rung-one row and the pricing problems are untouched. Given an incumbent plan with fleet vector $\mathbf{n}^\star$ and total $X^\star$, the tree has two levels that ask different questions — level 1 on the total, level 2 on the vector — and they are exactly the two aggregates of rung one, in the order the paragraph above puts them. Figure 5 draws them. The delivery rung is kept as a third level of the tree, entered only where the first two have nothing left to say: a region that fixes the whole fleet vector and whose relaxation is still fractional.

Level 1: the total fleet. A bound on the total fleet has nothing to compensate it, as above. Three regions partition the space:

- $X \ge X^\star + 1$ costs a whole truck with no offset. A plan in it improves on the incumbent only if the extra truck saves more than cK minus the incumbent's gap in inventory, and the region's relaxation says at once whether that is possible; it is usually pruned by bound.
- $X = X^\star$ fixes transport. Everything inside is inventory allocation, and this is where the second level goes.
- $X \le X^\star - 1$ is the only region that can beat the incumbent outright — "is there a plan with fewer trucks?" — and is therefore worth a certificate of its own. It is also the region the tree constructs in *first*: with the equality region on top of the stack its Murty subtree can take the whole budget while a smaller fleet is never asked for, which is what happened on a pure pool before the order was changed (Table 8).

A plan found *inside* an inequality region, with total X_2, splits that region on its total exactly as the root was split — $[\,\cdot, X_2 - 1]$, $\{X_2\}$ and $[X_2 + 1, \cdot\,]$ cover it — and the equality piece starts a Murty chain of its own; level 1 is recursive. A construction that fails inside a region says only that the *pool* cannot support that fleet, since step 2 is exact over the pool; the region is therefore priced under its own duals with a small share of the budget and constructed once more before it is given up, and a bound proved by that pricing prunes it.

Level 2: the fleet vector, inside $X = X^\star$. Fix an order on the pairs (g, t) — periods first, then direction. The fleet vector of any plan in the region differs from $\mathbf{n}^\star$ at a first index

j, or nowhere. Branching at the first pair where a solution may differ gives three children, $n_j \le n_j^\star - 1$, $n_j \ge n_j^\star + 1$ and $n_j = n_j^\star$, and the equality child is partitioned again at index $j+1$ (Murty's scheme). The regions are disjoint and exhaustive, and the equality child is expanded first so that the neighbourhood of the incumbent is searched before its complement. The partition is taken around the plan *constructed at that node*, not around the global incumbent, so a node that improves the incumbent is immediately re-partitioned around the better plan and nothing has to be restarted. The chain is cut at a depth that rises by two whenever the stack empties with budget left.

Processing a node. A node is a region imposed on the RMP as fleet rows. The LP over the current pool is solved first; being an over-estimate of the region's LP it is a filter and never a bound (Section 2.5). If the filter allows pruning and the certification policy admits the node, column generation is run to a certificate; a certified value at or above the incumbent prunes the node, an infeasible region is discarded, and a certified value below the incumbent is recorded as that region's bound. In every other case the node carries its parent's bound, which remains valid for the child because the child's region is a subset of the parent's. The flow-first relax and fix of Section 2.8.1 is then run inside the region, and if it returns a plan cheaper than the incumbent the incumbent is replaced. Finally, if the region fixes X, the Murty children are generated around the constructed plan.

Two generators, one pool. Pricing returns extreme points of $\mathrm{conv}(\Omega^g)$; the relax and fix returns covers of a delivery vector, which are interior points that no dual vector reaches (Figure 4). A column is a feasible load, and a node's constraints bound the multipliers λ, not the load, so a column found anywhere is valid everywhere: every column from either generator goes into the one shared pool, and a node that is pruned still leaves its columns behind. That is the mechanism by which the tree improves on the root. It is not that the tree reaches an integral relaxation — it essentially never does — but that each region it visits asks the construction for a plan under constraints the root never saw, and the loads that plan needs are added for every later node to use.

What the tree reports. The incumbent is the cheapest verified plan found by any construction. The root bound is the certificate obtained before any branching — under 3D loading, that of the volumetric relaxation (Proposition 2). The global bound of a branch and bound is the minimum over its *frontier*, not its root: a region nobody expanded contributes nothing, which is why a dive reports its root value however far it has gone. At the end of the budget the regions still open are therefore swept — filtered by their pool LP and certified where the filter allows — and the global bound is the minimum over the certified bounds of the open regions, floored at the root bound and capped at the incumbent. Under 3D loading a region's certificate is taken, like the root's, from the volumetric relaxation with the region's fleet rows applied, which Proposition 2 covers: the rows bound the fleet vector, not the load model, so the volumetric master over the region bounds the 3D master over it. Optimality is claimed *only* when the root is certified, no region remains open, no restart has broken the partition, and the incumbent meets the global bound. No run reported in Section 3 meets these conditions, so the method certifies no optimum anywhere in this paper; the one optimum the paper does report is established by inclusion from an exactly solved relaxation and not by the method (Section 3.4).

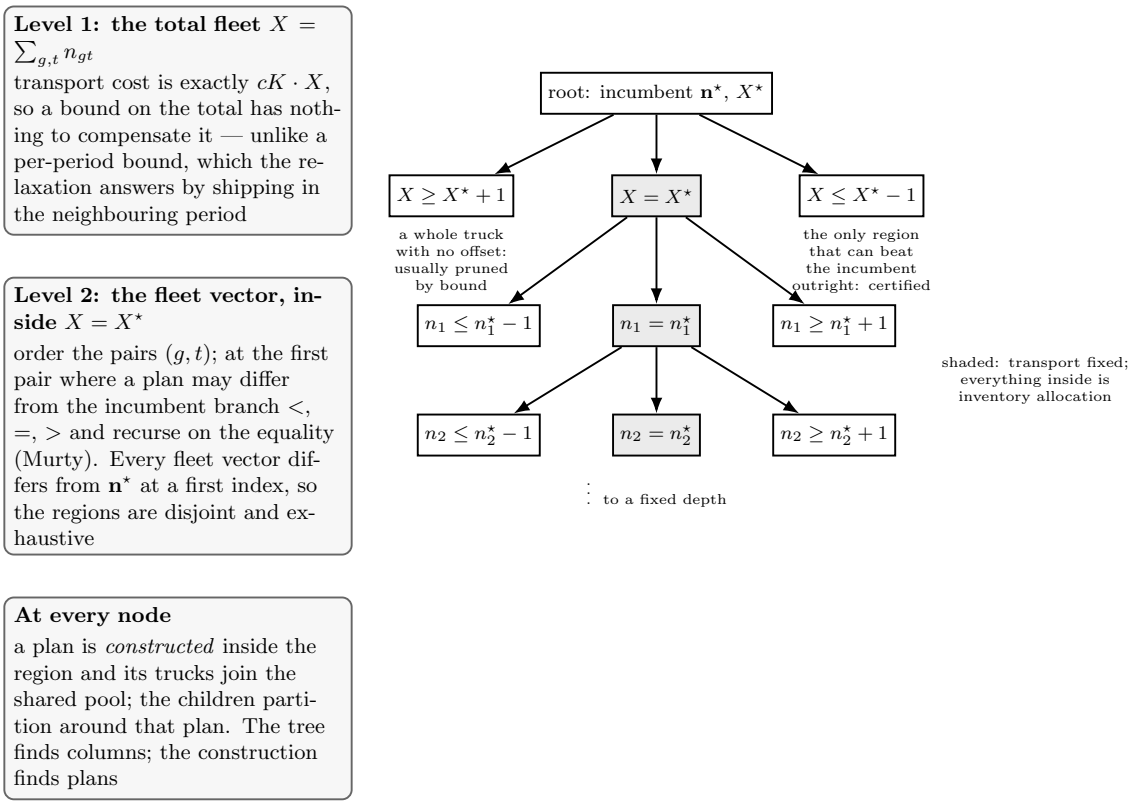


Figure 5: The two levels of the fleet tree. Level 1 partitions on the total fleet X, whose cost $cK \cdot X$ the relaxation cannot offset; level 2 partitions the region $X = X^\star$ on the fleet vector itself, one pair at a time around the plan constructed at each node. Bounds on X and on n_{gt} are rows of the master that already carry duals, so the pricing subproblem is untouched, and a column found in any region is valid in every other.

2.10 The construction under 3D loading

Step 3 of Section 2.8.1 builds trucks from a delivery vector with a volumetric notion of a legal load: (86)–(87) are exactly the volumetric column conditions. Over the volumetric oracle that notion *is* the definition of a column, and every part the partition proposes is a column by construction. Under 3D loading a load must also satisfy (49)–(54). The volumetric conditions are nevertheless *necessary* for geometric feasibility (Proposition 2), so the partition programme is kept as a *relaxation* of the geometric partition and each part it proposes is then tested for placeability. Step 2 makes no oracle call and is unchanged, so the objective is still decided before any geometry is examined; what changes is only whether the trucks that objective implies can be built.

Deciding placeability. A part is submitted to a placeability oracle that returns one of three answers. A constructive *shelf design* — a small integer programme that lays the part's stacks in shelves along the trailer length, one integer count per shelf, footprint and orientation — supplies essentially every positive answer, and its layout is passed through the independent verifier of Section 3 before it counts; a witness the verifier rejects is not a witness. The pricing problem of Section 2.6, run in feasibility mode with the true slot bound (48), supplies the *infeasibility* proofs; the true bound is what makes an infeasible verdict a proof, since a model with fewer slots than a layout needs can report infeasible while the load is placeable. A part

left undecided within the limit is neither accepted nor refuted. A placeable part becomes a column with its layout as witness, exactly as a priced column does.

Two kinds of cut, and they are different claims. A part *proved* unplaceable yields a *dominance cut* forbidding every load that dominates it componentwise. The cut is valid because placeability is downward closed: removing an RTI from a placeable layout leaves every position unchanged and every stack no taller and no heavier, so a load dominating an unplaceable one is itself unplaceable, and the cut removes the whole up-set of the refuted load rather than a single point. The full-truckload floor is not part of the placeability test, so it does not break the closure. A part left *undecided* is instead excluded as a single point, so that the partition proposes something else. That is a restriction of a primal heuristic and never a validity claim: it cannot make the method report an invalid plan, only fail to find one. The two constraints are kept separate because they rest on different grounds, and only the first may ever be read as knowledge about the instance.

A dominance cut written at the refuted load itself is weaker than it needs to be, and the difference matters on a multi-commodity part. Before cutting, the refuted load is tightened *along its ray*: $\lfloor \sigma v \rfloor$ is probed for σ bisected in $(0, 1]$, and the cut is written at the smallest scaled-down point still *proved* unplaceable. Every point of the one-unit-at-a-time staircase the partition would otherwise walk down — $\{R_1 : 272\}$, 271, 270, ... on one measured pair — dominates that point, so one cut removes the staircase instead of its top step. The bisection is seven probes at most, its placeable verdicts are cached like any other, and an *undecided* probe stops it at the last proof, so the cut is never written anywhere that was not proved. Without this step a reimplementation reproduces the staircase and exhausts its round cap without reaching the answer.

Placeability caps. The maximum count of a single commodity in one truck is a scalar, and by the same closure it is monotone, so it is bisected once per direction and commodity at the start of a run — a proved pricing optimum tightens it, an undecided probe leaves the analytic cap in force — and handed to the partition as the bound $\bar{u}_{gc}$ before it solves. Without it a single-commodity dominance cut advances one unit per round, from the volumetric fill down towards the placeable count, at a placeability call and a partition re-solve each. A cap is also a measurement in its own right: it is the largest number of one commodity that *can* be placed, and setting it beside the number a volumetric capacity credits measures the relaxation's optimism directly, with no plan involved. Section 3.7 reports that comparison.

A stack relaxation inside the partition. The partition additionally carries, per truck and footprint, an integer number of stacks, with the crush limit and the height budget imposed per stack and the stacks' floor areas summed against the trailer floor; summed over the n_{gt} trucks of a period the same rows sit inside the aggregate master of step 2, with one integer count per footprint and period. The rows are necessary — every truck of the period has its own integer stack counts satisfying the per-truck rows, and summing them over the period's trucks gives exactly the aggregate rows — and they are precisely what the volumetric relaxation omits: a family whose RTIs are short enough for four layers by height but heavy enough for only two by the crush limit is credited by volume with twice its placeable count. The layer coefficient must be the *binding* one, the smaller of the height-limited and crush-limited counts; written with the height count alone the row is not a necessary condition and cuts off feasible fleets. The relaxation says nothing about the two-dimensional arrangement of the stacks, which the placeability test and the dominance cuts cover.

The pool and the bound. The starting pool is the placeable single-commodity trucks of Section 2.7 and nothing else; every other column comes from the pricing ladder or from the relax and fix. On a catalogue where the crush limit binds, the restricted master over that pool is not even feasible — no combination of pure trucks covers demand within the fleet — so clearing the artificials is the first thing column generation has to do. *Either* generator does it: at the per-call budget of Section 3 the exact oracle alone clears them in one sweep, and so does one rung-fed sweep. What the rung buys is not feasibility but a tighter restricted master per unit of work, and Section 3.5 measures that (Table 9). The root bound is the certified value of the volumetric relaxation (Proposition 2); geometric pricing, given a share of the root budget, supplies columns and nothing else. Section 3 reports what it supplies.

2.11 Two corrections to the compact model

Implementing 3D-SCPTOP surfaced two points worth recording, both invisible at $lt = 1$.

Inventory recursion under a lead time. Constraints (9) and (16) carry inventory forward from period $t-1$ and delay only the *arrival*, which is what the physical system does. An earlier statement of the model carried the stock itself forward from $t - lt$,

$$I_{pnt} = I_{p,n,t-lt} - d_{pnt} + \overline{V}_{n,t-lt} \qquad \forall n \in N,\ t \geq lt + 1, \tag{89}$$

$$Ie_{str} = Ie_{s,t-lt,r} + \overline{V^{\mathrm{e}}}_{r,t-lt} - \textstyle\sum_{n \in M(r)} d_{snt} \qquad \forall r \in R,\ t \geq lt + 1, \tag{90}$$

which silently discards $lt - 1$ periods of stock. With $lt = 1$ the two versions coincide, which is why the defect survived; every model in this paper is run at $lt = 1$, so no reported number changes either way. The correction is recorded because the formulation is the contribution.

Coverage stock on folded RTIs. Constraint (20) requires $Ie_{gtr} \geq sse_{gtr}$ at *both* nodes, and every model in this paper enforces it that way: the master of Section 2.3, the volumetric benchmark of Appendix **??**, the company procedure PLC-SCPTOP and the compact model itself. An earlier implementation imposed the folded bound at the supplier only, on the argument that the plant has no operational use for a stock of empties and that the extra row could render an instance with a tight RTI fleet infeasible. The row is what the model states, so it was reinstated, and the concern did not materialise: on the largest instances of Section 3 the root LP value is identical with and without it, no coverage row carries artificial slack, and the row is tight in a minority of periods without moving the optimum. Enforcing it everywhere also removes an asymmetry: had the master enforced the plant side and the comparison models not, the comparison would have been unfair in the master's favour.

3 Computational Experiments

The experiments answer three questions, and the order matters because the first is what licenses the other two.

1. **Does the reformulation find good plans where the truth is known?** The volumetric problem of Appendix **??** is the same transport and inventory problem with a volumetric truck capacity, and the compact formulation of it is solvable on instances of the size this study reaches. It is therefore a genuine yardstick, and Section 3.4 sets the reformulation against it.

2. **Does the method keep working on the 3D problem, where no reference optimum is available?** The compact 3D model returns no incumbent beyond five periods, so it cannot serve as a yardstick; Section 3.5 reports what the method returns and what it can certify, and Section 3.6 states what remains to be measured against the procedure the company uses today.

3. **Does it matter?** Section 3.7 puts the loads a volumetric model prescribes to an independent placeability proof, which is the test of whether the 3D representation buys anything at all.

3.1 Protocol and declared budgets

All calculations use Python. The restricted master, the aggregate integer master and the partitions of Section 2.8.1, the volumetric benchmark and the compact models are solved with Gurobi 12; the pricing problems (61)–(83) and the placeability tests with CP-SAT from OR-Tools 9.12, which solves the equivalent statement of the same feasible set described in Section 2.6. The solver is how these numbers were obtained and is not part of the method: the method asks the oracle for an optimal point of the pricing problem, and any solver that returns one serves. Every run reported in Tables 5–10 is read from a recorded run file, named in a comment beside the row in the source of the table.

Every row carries its configuration, or says that it does not. Five parameters can change the answer a solver returns: the wall-clock *time limit*; the relative *MIP gap* at which an integer solve may stop; the *thread count*; the *memory cap* per model; and the random *seed*. Table 4 states them per stage. They are read from each run's own record and not from the script that launched it — with one exception, which is marked wherever it appears. The volumetric fleet-tree runs of Table 5 recorded no configuration, and theirs is *recovered* after the fact from their launcher scripts and the system journal; one of those five runs has no recoverable time limit at all. A recovered configuration is weaker evidence than a recorded one and is never silently mixed with it.

Memory is a budget, not an accident. Gurobi holds its branch-and-bound tree in memory and imposes no limit of its own, so a compact model whose dual bound has stalled simply grows its list of unexplored nodes until the machine intervenes. Every model is therefore given an explicit per-model soft memory limit, at which the solver stops and returns its incumbent and its dual bound with the status MEM_LIMIT, exactly as a time limit would. Two consequences follow. A solve that stopped on memory is a solve that stopped, and its dual bound is a valid lower bound on the model's optimum regardless of why it stopped. And reaching a *fixed* per-model cap sooner on a larger model is a statement about the cap and not about tractability: no inference is drawn here from *when* the cap was reached, and the cap is reported as a parameter of the experiment like the time limit.

What each reported quantity means.

- z is the cost (55) of the plan returned, net of any artificial penalty; a plan is reported only if it carries no artificial slack. For the fleet tree it is the incumbent of Algorithm 1; for the compact models it is the solver's incumbent at its stop. It is an upper bound on the optimum in every case.

- *trucks* is $\sum_{g,t} n_{gt}$ for that plan.

Table 4: Declared budgets, per stage. The memory cap is Gurobi's per-model soft limit, at which a solve stops and keeps its incumbent and dual bound. Values marked ~ are recovered from a launcher script rather than recorded by the run (`results/hp/provenance.json`); the recovered memory cap is the implementation's 4 GB default, which applied because no launcher set the environment variable that overrides it and `systemd-run` does not inherit a terminal's environment. The value marked † is neither recorded nor asked for: the launcher asked for ten threads, but the implementation then built each Gurobi model on a fresh environment, which does not inherit a thread count set on the default one, so every model in those runs ran on all twelve cores of the machine. The parameter was threaded through on 2026-09-14, after those runs. The fleet tree divides its time limit between phases as described in Section 2.9: in 3D, 15% of the budget to root column generation, 3% reserved for the frontier sweep, placeability calls limited to 5 s and pricing calls to 20 s at eight CP-SAT workers each, the heuristic rung on with a 0.5% qualifying margin and the stop-early oracle and pre-empting the oracle at a pair it serves — which is what the budget condition of Section 2.6 selects at a 20 s per-call limit — an initial Murty depth of 6 raised by two whenever the stack empties with budget left, at most two cut-and-repack rounds per partition attempt, and at most 15% of the budget on region certificates. The 900 s budget applies to the thirty-six generated instances and to the first two rungs of the size ladder, 1 800 s to the third and fourth rungs and 3 600 s to the fifth.

stage	time limit (s)	MIP gap	threads	memory cap (GB)	seed
MATH-3D-SCPTOP, 36 generated instances (Table 11)	900	$5 \cdot 10^{-4}$ (step 2), 0 (step 4)	10	20	0
MATH-3D-SCPTOP, size ladder (Tables 6–9)	900 / 1 800 / 3 600	$5 \cdot 10^{-4}$ (step 2), 0 (step 4)	10	20	0
fleet tree, volumetric oracle (Table 5)	1 800~	10^{-4}	12† (10 asked)	4~	not established
compact SCPTOP, volumetric reference (Table 5)	1 800 / 3 600	0	10	20	0
compact 3D-SCPTOP, $n_{\mathrm{LC}} = 3$ (Table 7)	1 800	0	10	20	0
SCPTOP, volumetric benchmark (Tables 11–12)	900	0.02	4	20	0
placeability recheck of SCPTOP loads	120 per load	—	4	20	0

- LB is a lower bound on the optimum of the model the method solves. For the fleet tree it is the *certified* bound of Section 2.5: the root certificate, or the minimum over the certified bounds of the open frontier when that is higher. A bound is printed only where the sweep behind it came back with every pair proved; where it did not, the row says *unproved* and no gap is computed. For a compact model it is the solver's dual bound at its stop. A sentinel recorded by a run that obtained no bound is never printed, never averaged and never rounded to a side.

- *gap* is $(z - \mathrm{LB})/z$ against the method's own bound. It is an optimality gap only to the extent that the bound is tight, and Section 3.7 identifies exactly where it is not. It is never a gap to a proven optimum unless the row says so.

- *stop* names the condition under which a solve ended: OPTIMAL, TIME_LIMIT or MEM_LIMIT. A time or memory stop makes the row's time the budget rather than a solve time.

- *nodes* is the tree nodes opened and *open* the regions left unexpanded when time ran out; a run with open regions has not searched the space, whatever its gap.

- *verified* is the outcome of re-deriving every dispatched truck of a plan from the column's own fill and checking it against (49)–(54) — placement, stack height, crush limit, payload and the utilisation floor — by a routine independent of the search and of the oracle that built the column, which re-derives all of those from the stored layout rather than reading any quantity the builder recorded. It is not code-disjoint from the rest of the implementation: the full-truckload predicate *is* the one the oracle, the column reductions and the partition floor use, together with the two commodity-to-weight and commodity-

to-type lookups, so a defect in the *definition* of a full truck would not be caught by this check. What it catches is a defect in any solver's *satisfaction* of that definition, which is what it is there for. *unplaceable* counts, among the loads a *volumetric* plan dispatches, those a constraint programme has *proved* cannot be stacked and placed in a trailer; loads it could not decide within its limit are reported as *undecided* alongside, and are counted neither way.

- Δz is the cost of one method relative to another on the same instance. The denominator is stated wherever it appears, and Δz is meaningful only between plans both of which are executable.

3.2 Instance generation

Thirty-six benchmark instances are generated as a full factorial over the four explanatory variables of the study: the number of products $|N| \in \{4, 8\}$, the horizon $|T| \in \{5, 10\}$, the number of RTI types $|R| \in \{2, 3, 4\}$ and the volumetric granularity $v \in \{60, 90, 120\}$. Instances are named $I_|N|_|T|_|R|_v$.

- **Container dimensions**: footprints are drawn from the industrial catalogue $\{1200 \times 800,\ 1200 \times 1000,\ 800 \times 600,\ 1200 \times 1600,\ 2400 \times 1200\}$ mm. Heights are quantised to 50 mm and a footprint is admissible for a given v only if the implied erected height lies in $[400, 1500]$ mm, so that an RTI is worth stacking and at least two layers fit under the trailer roof. Whenever $|R| \geq 3$ several RTI types share a footprint, reproducing a real catalogue in which container families sit on a common pallet base; this is what allows a stack to mix types across layers.

- **Container volume**: v is the number of erected RTIs that fill one truck by volume, and every RTI type is generated with volume at least Γ_K / v. Larger v means smaller RTIs and a harder placement subproblem; smaller v means a single container is a larger fraction of a trailer.

- **Demand**: generated per product and period by deviating $\pm 10\%$ from a mean calibrated so that roughly 2.5 truckloads are consumed per period. Calibrating the mean keeps difficulty comparable across the factorial instead of letting it drift with the RTI size.

- **Weights**: the content weight of a full RTI scales with its volume and varies by $\pm 25\%$ between products sharing an RTI type, which is what makes the per-stack crush limit depend on the period's product mix rather than on the RTI type alone.

The truck is a standard semi-trailer, $pK \times qK \times rK = 13600 \times 2400 \times 3000$ mm, payload $pK^w = 24$ t, per-stack crush limit $W^{\max} = 1200$ kg, minimum utilisation $\alpha = 0.55$ and cost $cK = 1000$ per dispatch. Lead time is $lt = 1$ and coverage stock one period of demand.

A size ladder. The benchmark tops out at $|N| = 8$ and $|T| = 10$. To find where, if anywhere, the decomposition pays against the compact model, a second set of instances is generated with the same rules along two axes: $(|N|, |T|) \in \{(8, 10), (12, 12), (16, 15), (24, 20)\}$, and the number of truckloads consumed per period, 2.5, 5 or 8, which scales the fleet K and with it the number of interchangeable trucks the compact model indexes. The suffix is that figure times ten and is *not* K: $I_8_10_2_90_k25$, $I_12_12_2_90_k25$ and $I_16_15_2_90_k25$ have $K = 12$, and $I_24_20_2_90_k50$ has $K = 21$. These instances have $|R| = 2$ and $v = 90$. They stress different things: $|N|$ and $|T|$ grow the master and the number of pricing calls, while the fleet grows the compact model's per-truck variables and its symmetry.

Sizing the RTI fleet. One generation parameter deserves comment because it is easy to get wrong and the symptom is misleading. Empties return by the truckload, and a return truck leaves only once the plant has accumulated enough folded RTIs to clear the full-truckload floor. Since folded RTIs pack very densely, that takes several periods, during which the supplier must live off its own folded stock. If the fleet is sized only for the lead time, the linear relaxation remains feasible by dispatching fractional trucks while the integer problem has no solution at all. We therefore size the supplier's initial folded stock of each type to cover $lt + SC$ periods plus the number of periods, capped at six, needed to accumulate $\lceil \alpha \Gamma_K / \gamma_{sr} \rceil$ folded RTIs at the plant's rate of emptying them, which is the standard closed-loop argument that an RTI fleet must cover the cycle time.

3.3 Methods compared, and which procedure produced which table

Three solution procedures appear in the recorded results of this study, and only the first is the method of Section 2. They are listed with the tables they produced, because a table labelled with the method's name but produced by another procedure is the single easiest way for a study like this one to mislead.

- **MATH-3D-SCPTOP** — Algorithm 1 exactly as Section 2 states it: the master of Section 2.3 over a starting pool of the placeable single-commodity trucks and nothing else, columns from the pricing ladder of Section 2.6, the flow-first relax and fix of Section 2.8.1 as primal heuristic with the partition run as a cover, every part proved placeable before it becomes a column, dominance cuts on the parts proved unplaceable, integer stack counts in the aggregate master, the region $X \leq X^\star - 1$ constructed first, and the bound from the volumetric relaxation (Proposition 2). Every dispatched truck is verified. *This produced Tables 11, 12 and 6–10, and every MATH-3D-SCPTOP number quoted in this paper.*

- **The same method over the volumetric oracle** — the same master, the same search and the same construction, with the pricing problem replaced by the two-constraint knapsack of Section 2.6, so that a column is a volumetrically feasible full truck. *This produced the fleet-tree columns of Table 5.*

- **An earlier branch-and-price configuration**, in which the primal heuristic is the integer master over a pool seeded with the volumetric benchmark's placeable loads rather than the flow-first construction. It is a different procedure — a different pool, a different primal, and no placeability cuts — and on $I_8_5_2_90$ it returns 20 767.81 with fourteen trucks where MATH-3D-SCPTOP returns 18 796.96 with twelve. *No table in this paper reports it.* Its recorded output, `results/benchmark.csv` and `results/case_study.csv`, is not used for any method column here; the volumetric half of the first of those files, which involves no branch and price at all, is used for the placeability census of Section 3.7 and for nothing else.

The comparators are:

- **Compact SCPTOP** — the volumetric model of Appendix **??** solved directly by Gurobi. It solves the same problem as the fleet tree over the volumetric oracle, so the two bracket a common optimum, and it is the one comparator in this study that produces a *proved* optimum.

- **Compact 3D-SCPTOP** — the model of Section 1 solved directly by Gurobi, warm-started with the matheuristic's plan. Section 1 lets every truck design its own layout, which puts a complete packing problem on each of the $2|K||T|$ trucks; what is solved here instead shares a catalogue of $n_{\mathrm{LC}} = 3$ layouts per direction among them, so that the trucks of a direction must agree on three designs. That is a *restriction* of Section 1, chosen a priori: nothing here computes an over-estimate of how many designs suffice. Its *plan* is therefore a legitimate comparator — a feasible plan of the restriction is a feasible plan of the model — and its dual bound bounds the restriction and is not an end of any interval for $z^\star$.
- **SCPTOP** — the volumetric benchmark, identical to the master in every respect *except* that truck capacity is volumetric. It is not used as a yardstick for the 3D method; its role is Section 3.7, where the loads it dispatches are put to a placeability proof.
- **Company procedure (PLC-SCPTOP)** — the master over a fixed pool of manually derived loading configurations, as produced today by a spreadsheet. This is the intended comparator for the 3D rows, and Section 3.6 states why no number is reported against it here.

3.4 The volumetric model against the compact SCPTOP formulation

Table 5: The reformulation against the compact SCPTOP formulation, both solving the *volumetric* problem of Appendix **??**, so that the two bracket one common optimum. This is the comparison in which a reference optimum exists: the compact model closes two of the six instances. z is the incumbent at the stop; OPT marks a proved optimum, ^{m} a stop on the memory cap and ^{t} a stop on time. "bound" is Gurobi's dual bound for the compact model and the certified master-LP bound of Section 2.5 for the fleet tree. Δz is $(z_{\mathrm{tree}} - z_{\mathrm{compact}})/z_{\mathrm{compact}}$; negative means the fleet tree's plan is cheaper. **The budgets are not equal.** The compact runs record their own configuration and had 3 600 s and a 20 GB per-model cap; the fleet-tree runs recorded none, and their configuration is *recovered* after the fact from their launchers (marked ~) as 1 800 s with the 4 GB default cap, with no seed or commit established and, on the last row, no time limit established either (marked $^{!}$). On time and memory the asymmetry runs against the fleet tree on every row where its budget is known. On cores it runs the other way, and the table marks it †: the launcher asked for ten threads, but the implementation built each model on a fresh Gurobi environment, which does not inherit that setting, so these runs used all twelve cores of the machine against the compact model's ten. A row on which the fleet tree is cheaper is therefore not a time or a memory artefact, but neither is any row here a timing claim; a row on which it is dearer may be a budget artefact.

	compact SCPTOP				fleet tree (volumetric oracle)					
Instance	z	bound	trucks	t (s)	z	bound (cert.)	trucks	t (s)	Δz (%)	tighter bound
I_8_5_2_90	18 796.96 OPT	18 796.96	12	0.2	*not run*	—	—	—	—	—
I_8_10_2_90_k25	38 855.65^{m}	38 851.93	26	3243	38 853.39^{t}	38 791.14	26	1817†	−0.0058	38 851.93 (compact)
I_16_15_2_90_k25	68 139.17 OPT	68 139.17	45	1349	68 139.59^{t}	68 131.43	45	1897†	+0.0006	68 139.17 (compact)
I_16_15_2_90_k50	134 346.91^{m}	133 417.47	89	1113	133 503.74^{t}	133 484.32	88	1827†	−0.6276	133 484.32 (DW)
I_24_20_2_90_k50	200 266.21^{m}	199 518.04	137	2088	200 047.54^{t}	200 013.41	137	1854†	−0.1092	200 013.41 (DW)
I_24_20_2_90_k80	282 101.77^{m}	280 823.53	189	2399	281 023.77^{t}	280 910.02	188	868$^{!\dagger}$	−0.3821	280 910.02 (DW)

Table 5 is the comparison in which the truth is sometimes known. On the volumetric problem the compact formulation is tractable at the sizes this study reaches — it closes two of the six instances to proved optimality — so the reformulation can be measured against a reference rather than against itself.

Against a proved optimum. On $I_16_15_2_90_k25$ the compact model proves the optimum, 68 139.17 with forty-five trucks, in 1 349 s of a 3 600 s budget at a 20 GB cap. The fleet tree over the volumetric oracle returns 68 139.59, also with forty-five trucks: 0.0006% above a *proved* optimum, and at the same fleet. Its own certified bound on the same instance, 68 131.43, lies 0.0114% below that optimum, so both ends of the reformulation's interval are within about a hundredth of a percent of the truth. This is the strongest credibility datum in the paper, and the reason for reporting it first is that nothing elsewhere in the study can be checked this way.

Four qualifications belong with it rather than after it, and the last two are the ones that limit what it proves.

What is compared here are two *volumetric* plans, and neither is executable as a 3D plan. That is the point of the stage — it measures the method against a reference optimum of the model the reference can solve — but it means the phrase "executable by construction" does not apply to this row. A column of the volumetric oracle is a load vector with no layout, so there is nothing for the verifier of Section 3.8 to re-derive, and no plan in Table 5 is verified truck by truck. How far such a plan is from executable is measured in Section 3.7 on the compact model's own optimum for this instance, where 41 of its 45 dispatched loads are *proved* unplaceable; the fleet tree's 45-truck volumetric plan at 68 139.59 has not been put through that census, and nothing here should be read as a claim that it would fare better.

The budgets are *not* equal. On time and memory the inequality runs against the fleet tree: the compact model had twice the time and five times the memory. On cores it runs the other way, and that has to be said in the same breath. The launcher asked for ten threads, but the tree built its Gurobi models on a fresh environment, which does not inherit a thread count set on the default one, so every model in those runs used all twelve cores of the machine against the compact model's ten; the parameter was only threaded through on 2026-09-14, five days after these runs. One resource axis therefore favoured the tree by a fifth of the cores, which is a further reason to read this row as an objective comparison and not as a timing claim.

The fleet-tree run recorded no configuration of its own, so the 1 800 s and the 4 GB in the table are recovered from its launcher and the system journal, with no seed and no commit established; a recovered configuration is weaker than a recorded one, and what it supports is the objective comparison — two numbers on one instance — and not a timing claim.

And the five volumetric runs are five days older than the 3D campaign: they ran the same *scheme* — the master of Section 2.3, the pure single-commodity pool, the flow-first relax and fix, the two-level fleet branch and bound — but an earlier revision of it, in which step 3 solved an exact split rather than the cover of Section 2.8.1, step 2 carried no integer stack counts, and level 1 did not recurse. They are therefore evidence about the scheme and not a measurement of the code that produced Table 6. Re-running them on the current code is a cheap and obvious next step, and until it is done this row is the weakest-provenanced of the paper's strong claims.

One thing the volumetric rows do measure cleanly is the tree, because there it certifies per node. On $I_16_15_2_90_k25$ its root construction returns 68 151.00 at 332 s and four node improvements take that to 68 139.59, a gain of 0.0167% that lands within 0.0006% of the proved optimum. The pattern repeats on the three larger rows: root plans of 133 506.22, 200 053.56 and 281 024.98 improved by three, five and two node improvements to 133 503.74, 200 047.54 and 281 023.77 — gains of 0.0019%, 0.0030% and 0.0004%. The tree earns little here and much on the 3D rows, which is consistent with what the root leaves it: where the relaxation is nearly exact the construction is already there.

Where the compact model stops on memory. On the other four instances the compact model exhausts its 20 GB cap with its dual bound still short of its incumbent. The fleet tree's plan is cheaper on all four — by 0.0058%, 0.6276%, 0.1092% and 0.3821% of the compact

incumbent — and uses one truck fewer on two of them. Its certified bound is the tighter of the two on three of the four; on $I_8_10_2_90_k25$ the compact model's dual bound, 38 851.93, is tighter than the reformulation's 38 791.14 by 0.157%. The budget asymmetry means a row on which the fleet tree is cheaper is not a budget artefact, while the one row on which its bound is looser may be.

The bound, tighter and sooner. The clearest statement about the bound does not need the recovered runs. On $I_12_12_2_90_k25$ the reformulation certifies 52 105.25 after 26.3 s of its 1 800 s 3D run — a full sweep in which every (g, t) pair priced optimal and nothing priced out, over the volumetric relaxation and so valid for the 3D problem by Proposition 2. The compact volumetric model on the same instance, given 600 s, ten threads and a 20 GB cap, stops at a dual bound of 51 944.22 with an incumbent of 52 116.69. The decomposition's bound is 0.310% tighter and arrives in a twenty-third of the time. The two budgets differ and are stated for that reason; both bounds are valid bounds on the same relaxation.

The horizon walk, measured once. The volumetric problem is also where the second relax and fix of Section 2.8.2 has its recorded results, and they are reported here rather than beside the procedure. Eight runs on $I_24_20_2_90_k50$ at a 600 s budget, over the volumetric master and final pools of 551 to 3 518 columns over all eight: *every* incumbent any of the eight produced came from the walk, and none from the closing integer master over the whole horizon, which is the obstruction the walk exists to get round. The best was 206 492.17 with 137 trucks against a certified bound of 200 013.36 — a gap of 3.14% of the incumbent, on the definition of Section 3.1. Under period scoping and without the pricing repair, two of the eight returned no plan at all.

These runs are not comparable with the fleet-tree rows above: a different budget, a different machine configuration, and a procedure that was never asked for the same thing. What they establish is the one structural fact the walk was built for — that at twenty periods the monolithic integer master produces nothing and a committed prefix produces something — and nothing about which primal is better.

3.5 The 3D problem

The compact 3D model cannot serve as a yardstick. Table 7 sets the compact 3D-SCPTOP model against the method on four rungs of the ladder, and the finding is a negative one that the rest of this section depends on. On $I_8_5_2_90$ — the one instance on which the compact model returns a plan — it has three shared layouts per direction, 67 674 variables and 92 092 constraints, and stops on its 1 800 s time limit at 18 816.13 with a dual bound of 18 795.76, no phantom freight, twelve loads that pass the independent verifier, and a peak resident set of 3.8 GB against the 20 GB cap. The method reaches 18 796.96 with twelve verified trucks at 140.2 s, a plan 0.102% cheaper, which the 67 nodes the tree then opens do not improve.

From ten periods on the compact model returns *no feasible solution at all.* On $I_8_10_2_90_k25$ (119 424 variables, 164 692 constraints at $n_{\mathrm{LC}} = 3$), on $I_12_12_2_90_k25$ (200 748 and 256 180) and on $I_16_15_2_90_k25$ (322 734 and 393 412) it finds none in 1 800 s. All three stopped on time, not on memory: their peak resident sets were 6.1, 2.7 and 4.5 GB against the declared 20 GB cap, so the runs had their memory budget and did not use it, and the honest statement is that the model could not reach a first incumbent in the time it was given. Nor is this a guard rejecting a corrupt solution: the leak counter is zero because there was no solution to leak. Four rungs are four rungs and we do not generalise beyond the sizes they cover; what

Table 6: MATH-3D-SCPTOP on the size ladder, **every row on one commit** (`73411b9`; the optimality proof of footnote [b] is a separate run of the *compact volumetric* model, named in the source), ten threads, a 20 GB per-model cap and seed 0, run sequentially. *nodes* is the tree nodes opened, *dropped* the regions left with no bound covering them, *open* those still on the stack at the deadline; a region counted *closed* carries a certified bound, which is not the same as being refuted (see Section 3). *root* is the plan the construction returns before any node is popped and *tree* the improvement the search then makes on it, both read from the same run's trace: this is the controlled measurement of what branching contributes, and it replaces a cross-commit comparison of two verified upper bounds, which is not one. All plans are verified truck by truck by code sharing nothing with the solver. The bound is the certified volumetric relaxation, valid for 3D by Proposition 2; it is loose wherever the crush limit binds, so the gap on the last two rows is not an optimality gap. The LB and *gap* columns are the *method's own* certificate on every row, never a mixture, so that no row carries two gaps; where a tighter valid bound exists it is reported in Table 5 and noted in footnote [c]. *I*_8_5_2_90 is the paper's only proved optimum and the proof is not the method's: footnote [b].

Instance	z	trucks	verified	LB (cert.)	gap	root (t)	tree	nodes	dropped	open	wall / budget (s)
*I*_8_5_2_90	**18 796.96**[b]	12	12 / 12	18 795.54	0.0076%	18 796.96 (140 s)	0.00%	67	0	0	873 / 900
*I*_8_10_2_90_*k*25	38 855.34	26	26 / 26	38 791.14[c]	0.165%	38 865.30 (136 s)	0.03%	68	0	35	883 / 900
*I*_12_12_2_90_*k*25	52 129.45	34	34 / 34	52 105.25	0.046%	53 195.84 (459 s)	**2.00%**[d]	11	0	10	1 761 / 1 800
*I*_16_15_2_90_*k*25	78 549.14	54	54 / 54	68 131.43[c]	13.3%[e]	78 549.14 (1 181 s)	0.00%	1	0	2	1 746 / 1 800
*I*_24_20_2_90_*k*50	230 594.36	160	160 / 160	200 013.41	13.3%[e]	230 594.36 (1 418 s)	0.00%	2	0	4	3 492 / 3 600

[b] **This plan is optimal**, and the LB column does not say so because that column is the method's own certificate throughout. The proof is by inclusion: the compact volumetric relaxation of this instance solves to proved optimality at 18 796.96 in 0.18 s, and the verified 3D plan costs the same to machine precision. Every placeable load is volumetrically feasible, so $z^\star_{\mathrm{3D}} \geq z^\star_{\mathrm{vol}}$ and a feasible 3D plan attaining it is optimal. *This proof does not come from the method*: no tree, no pricing, no geometric certification. It is available because on this catalogue the height-limited and crush-limited layer counts coincide, which is precisely the condition under which Section 3 argues a volumetric capacity is adequate. The method's own attempt at the same proof — certifying, under their own duals, the three regions its search could not refute — failed, at 237 s per region. [c] On these two instances the compact volumetric model's dual bound is *tighter* than the method's own and is equally valid for 3D by inclusion — 38 851.93 at a memory stop and 68 139.17 proved optimal — which would narrow the gaps to 0.009% and 13.25%. It is reported in Table 5 and used as the interval end in Table 7, and deliberately *not* used here: the LB and gap columns of this table are the method's own certificate on every row, so that no row of it carries two bounds or two gaps. [d] This is where branching does the most work in the paper: the construction returns 53 195.84 at 459 s and five node improvements take it to 52 129.45, a 2.00% gain that reproduces under a second certification policy (53 248.76 → 52 127.89). A run on an earlier revision reached 52 123.03, 0.012% below this row; that is a comparison of two verified upper bounds across commits and we report it here rather than in the table, because it is not a controlled experiment. [e] Not an optimality gap. The volumetric bound is loose exactly where the crush limit halves the layer count, which is the regime these two instances were generated to expose.

Table 7: The compact 3D-SCPTOP model ($|L| = 3$ configurations per direction, Gurobi, 1 800 s) against MATH-3D-SCPTOP on the size ladder, the method's rows being those of Table 6. All runs used ten threads, a 20 GB per-model cap and seed 0. **The budgets differ on three rows**: the compact model had 1 800 s throughout, the method 900 s on the first two rows, 1 800 s on the next two — which are therefore the equal-budget pairs — and 3 600 s on the last. *stop* is how the compact solve ended and LB its dual bound there; *none* means no feasible solution was found inside the budget. The method's LB is its certified volumetric bound (Proposition 2), *gap* is to that bound, *first hit* the time at which the reported plan was found, and *verified* the independent truck-by-truck check. No row is an optimum except where footnote [b] applies. The last column is the interval in which $z^\star$ lies, taking the method's plan as the upper end and, as the lower end, the tighter bound *that is a bound on the problem*: the compact 3D model's dual bound bounds its own $|L| = 3$ restriction and not $z^\star$ (Proposition 1), so it appears in its column and never as an interval end, and where a compact *volumetric* solve exists its dual bound is used instead, being valid for 3D by inclusion.

	compact 3D-SCPTOP, 1 800 s						MATH-3D-SCPTOP							
Instance	z	trucks	LB	stop	RSS (GB)	verified	z	trucks	LB (cert.)	gap	budget (s)	first hit (s)	nodes / open	$z^\star$ in
I_8_5_2_90	18 816.13	12	18 795.76	TIME_LIMIT	3.8	12 / 12[a]	18 796.96	12	18 795.54	0.0076%	900	140.2	67 / 0	[18 796.96, 18 796.96][b]
I_8_10_2_90_k25	*none*	—	36 458.75	TIME_LIMIT	6.1	—	38 855.34	26	38 791.14	0.165%	900	590.7	68 / 35	[38 851.93, 38 855.34][c]
I_12_12_2_90_k25	*none*	—	48 516.86	TIME_LIMIT	2.7	—	52 129.45	34	52 105.25	0.046%	1 800	1 648.5	11 / 10	[52 105.25, 52 129.45]
I_16_15_2_90_k25	*none*	—	64 053.23	TIME_LIMIT	4.5	—	78 549.14	54	68 131.43[d]	13.3%[d]	1 800	1 180.9	1 / 2	[68 139.17, 78 549.14][c]
I_24_20_2_90_k50	not run						230 594.36	160	200 013.41[d]	13.3%[d]	3 600	1 418.2	2 / 4	[200 013.41, 230 594.36]

[a] From the re-run of the compact model after a defect in the layout extraction was fixed; the first run returned the same objective, bound and stop condition, with no phantom freight, but its extracted layouts stated under-filled slots against the designed layer count and failed the verifier. The re-run's twelve loads pass it. [b] The only optimum this paper reports, and it is not the method's: the compact volumetric model solves this instance to proved optimality at 18 796.96 in 0.18 s, every placeable load is volumetrically feasible, and the verified 3D plan attains that value (Section 3.5). [c] Lower end from the compact volumetric model's bound on the same instance, valid for 3D by Proposition 2 and tighter than the method's own: 38 851.93 at a memory stop, and 68 139.17 proved optimal (Table 5). [d] Not an optimality gap. The volumetric bound is loose exactly where the crush limit halves a family's layer count, which is the regime these two instances were generated to expose (Section 3.7).

Table 8: What the starting pool, the search order and the frontier reserve cost (ten threads, seed 0). The first row seeded the pool by hand with demand-mix and pair fills and is *not* the method; it is the reference the pure-pool rows are measured against, and it ran at a 4 GB cap on an earlier commit, so it is not a controlled comparison with them. The pure rows start from the single-commodity trucks and nothing else. *integer stacks* is whether step 2 counts stacks per footprint in integers; *order* is which level-1 region the tree constructs in first; *reserve* is the share of the budget held back for the frontier sweep. Rows marked *final* are the final code, in which a region's certificate comes from the volumetric relaxation; on it the sweep takes seconds, so a 15% reserve is unused time, and 3% is the value the campaign of Tables 6 and 7 ran at. No row of this table *is* a row of that campaign: these are earlier commits, kept because they isolate one decision each, and they are not compared with it on time. Every plan is verified truck by truck; the bound is the certified volumetric relaxation in every row. *commit* is the code revision each row ran on, read from the run's own record: no two rows of this table share one, so no pair of rows is a controlled comparison of anything but the knob its row changes, and the 0.012% between the two $I_12_12_2_90_k25$ rows is not a run-to-run spread on one configuration. The measured spread on that instance is reported in Section 3.5.

Instance	pool	rung	integer stacks	order	reserve	cap (GB)	z	trucks	first hit (s)	budget (s)	nodes / open	LB (cert.)	commit
$I_8_5_2_90$	seeded by hand (reference)	no	no	$X = X^\star$ first	15%	4	18 796.96	12	48.5	900	219 / 0	18 795.54	`773f8cf`
$I_8_5_2_90$	pure	no	no	$X = X^\star$ first	15%	4	19 682.36	13	172.2	900	170 / 10	18 795.54	`5390e6c`
$I_8_5_2_90$	pure	yes	no	$X = X^\star$ first	15%	20	19 682.36	13	140.3	900	180 / 9	18 795.54	`1d4ccbe`
$I_8_5_2_90$	pure	yes	yes	$X \leq X^\star - 1$ first	15%	20	18 796.96	12	139.9	900	170 / 5	18 795.54	`bdd39ca`
$I_8_5_2_90$	pure, final	yes	yes	$X \leq X^\star - 1$ first	15%	20	18 796.96	12	140.2	900	135 / 4	18 795.54	`4fe6dc6`
$I_8_5_2_90$	pure, final	yes	yes	$X \leq X^\star - 1$ first	3%	20	18 796.96	12	185.4	900	147 / 7	18 795.54	`4fe6dc6`
$I_8_10_2_90_k25$	pure, final	yes	yes	$X \leq X^\star - 1$ first	15%	20	38 859.44	26	420.8	900	18 / 7	38 791.14	`4fe6dc6`
$I_8_10_2_90_k25$	pure, final	yes	yes	$X \leq X^\star - 1$ first	3%	20	38 855.70	26	481.9	900	30 / 5	38 791.14	`4fe6dc6`
$I_12_12_2_90_k25$	pure, final	yes	yes	$X \leq X^\star - 1$ first	15%	20	52 129.91	34	1 515.3	1 800	12 / 9	52 105.25	`4fe6dc6`
$I_12_12_2_90_k25$	pure, final	yes	yes	$X \leq X^\star - 1$ first	3%	20	52 123.03	34	1 691.9	1 800	17 / 11	52 105.25	`4fe6dc6`

they establish is that beyond five periods the compact 3D model supplies neither a plan to compare against nor a bound that bounds the problem — its dual bound bounds the three-layout restriction — which is why the comparator for the 3D rows has to be the procedure in use rather than the exact model.

One proved optimum, and it is not the method's. On $I_8_5_2_90$ the volumetric relaxation solves to proved optimality at 18 796.96 in 0.18 s, and the verified 3D plan costs the same to machine precision. Every placeable load is volumetrically feasible, so $z^\star_{\mathrm{3D}} \geq z^\star_{\mathrm{vol}}$, and a feasible 3D plan attaining the latter is optimal. That is the only optimum this paper reports, and it is established by inclusion from an exactly solved relaxation — no tree, no pricing, no geometric certification. The method's own attempt at the same proof failed: certifying, under their own duals, the three regions its search could not refute returned 18 795.57, 18 795.54 and 18 795.54 against an incumbent of 18 796.96, none of them proved, at 237 s per region. The reason is arithmetic rather than accidental — with the fleet fixed at twelve trucks, refuting a leaf needs a maximum reduced cost below roughly 0.12 per column, while pricing on the same instance was still returning columns an order of magnitude below that. The proof is available on this instance because its two container families have coinciding height-limited and crush-limited layer counts, which is precisely the condition under which Section 3.7 finds a volumetric capacity adequate.

The size ladder. Table 6 reports the method on five rungs, every row on one commit and verified truck by truck. Read as a progression it says three things, and the gaps have to be read with one convention fixed: the gap column of that table is to the *method's own* certified bound on every row, so that no row carries two gaps, and where the compact volumetric model

Table 9: The pricing ladder. **Top: three arms over one pure pool**, differing only in which columns enter it — the exact oracle alone, the heuristic rung pre-empting the oracle wherever it qualifies, and the two run additively. *LP* is the restricted-master value after the stated number of sweeps, and *lower is tighter*, since that value is an upper bound on the master LP until pricing converges. The 5 s blocks are a measurement of *which columns each generator finds*: the exact oracle is called at every pair in all three arms, so their walls differ only by the rung's own cost and are not production figures. The 20 s block is the production structure at the campaign's own per-call budget, where the pre-emptive arm does skip the oracle at a served pair, so its wall is comparable. The 5 s and 20 s jobs ran at 1.96 and ∼8 sampled cores respectively, so no row of one block is comparable with a row of the other. The exact-only arm's artificial-laden value at 5 s is a property of *that cap*: at 20 s the same arm reaches zero artificials at sweep one and finishes feasible, so it is not evidence that exact pricing cannot clear this master. **Bottom: the rung inside the one-commit production runs** of Table 6: pricing calls, calls the rung served without the exact oracle, columns it kept, exact-oracle calls, and the dispatched trucks of the reported plan (weighted by dispatch count) that are the rung's columns. It contributes no truck to the three plans whose certified gap is under 0.05%, and 44 of 54 and 122 of 160 trucks to the two plans on the crush-binding catalogue — which are also the two on which the pure pool *by itself* cannot cover demand within the fleet. No bound in this paper is the rung's: a sweep at which it served any pair cannot report *proved*.

Three arms over one pure pool (`ladder_both_*.json`, `ladder_exact_tl20_*.json`, `ladder_rung_tl20_*.json`)

Instance, per-call budget	arm	sweeps	LP	columns
$I_8_5_2_90$, 5 s	exact oracle only	5	18 796.61	30
	rung pre-empts	5	18 796.04	39
	rung and oracle	5	18 796.11	44
$I_16_15_2_90_k25$, 5 s	exact oracle only	3	$9.96 \cdot 10^7$ (99.6 artif.)	60
	rung pre-empts	3	76 839.25	103
	rung and oracle	3	76 304.92	148
$I_16_15_2_90_k25$, 20 s	exact oracle only	6	76 103.92	102
	rung pre-empts	6	75 983.75	185
	rung and oracle	6	75 361.05	265

Distinct commodities per column, both generators under one dual trajectory: oracle 4.72 *(median* 5*), rung* 10.17 *(medi*

In the production runs (`stats.ladder` of the Table 6 files)

Instance	calls	served by rung	kept	exact calls	rung tr
$I_8_5_2_90$	215	13	22	202	
$I_8_10_2_90_k25$	69	15	27	54	
$I_12_12_2_90_k25$	227	66	118	161	
$I_16_15_2_90_k25$	585	206	355	379	
$I_24_20_2_90_k50$	873	318	541	555	

Table 10: Fleet floors certified offline. The volumetric relaxation's master, restricted to $\sum_{g,t} n_{gt} \leq X$, is priced to convergence with one thread (commit dd5f039); INFEASIBLE means no plan with at most X trucks exists, in 3D as well, since every placeable load is volumetrically feasible and the fleet row does not depend on the load model (Proposition 2). A bound instead of a refutation means the relaxation admits plans with X trucks, so nothing about the truck count is proved. *time* is the region certification alone; the root certificate it rests on took 4 to 32 s. The two $I_16_15_2_90_k25$ rows were certified against an earlier fifty-three-truck plan; the plan reported in Table 6 has fifty-four trucks and $X \leq 53$ was not tested, so no claim is made about that instance's minimum fleet either way.

Instance	plan's trucks	X tested	verdict	bound over region	time (s)
$I_8_5_2_90$	12	≤ 11	INFEASIBLE: 12 is the minimum fleet	—	1.5
$I_8_10_2_90_k25$	26	≤ 25	INFEASIBLE: 26 is the minimum fleet	—	3.9
$I_12_12_2_90_k25$	34	≤ 33	INFEASIBLE: 34 is the minimum fleet	—	16.3
$I_16_15_2_90_k25$	54	≤ 52	not refuted	68 131.43	1.3
$I_16_15_2_90_k25$	54	≤ 45	not refuted	68 131.43	1.2

supplies a tighter valid bound it is reported in Table 5 and used as the interval end in Table 7, never as a second gap.

Against its own bound the gap is 0.0076% at five periods, 0.165% at ten and 0.046% at twelve — so two of the first three rungs are under 0.05% and the third is under 0.2% — and on all three the truck count is the minimum possible, because the region with one truck fewer is certified infeasible (Table 10): $X \leq 11$, $X \leq 25$ and $X \leq 33$ are infeasible in 1.5, 3.9 and 16.3 s, and the reported plans use twelve, twenty-six and thirty-four trucks. What remains of those three gaps is inventory cost. At fifteen and twenty periods the method still returns verified plans — fifty-four trucks at 78 549.14 and one hundred and sixty at 230 594.36 — but the gap jumps to 13.3%, and Section 3.7 attributes that to the bound rather than to the plan. Over the generated set the mean of the same gap is 8.8%, taken over the thirty of thirty-six instances that carry both a plan and a proved bound, and it reaches 30.72% (Table 12, whose "(30)" is that denominator); nothing in this paper supports a blanket statement that the method's gaps are small, and the one that can be made is the conditional one: they are small where the volumetric relaxation is nearly exact, and that condition is checkable in advance (Section 3.7). On $I_16_15_2_90_k25$ neither $X \leq 52$ nor $X \leq 45$ is refuted, so nothing about the truck count is proved there, for the same reason the bound is loose: the volumetric relaxation admits forty-five-truck plans — its own optimum has forty-five — and cannot separate the fifty-four-truck 3D plan from fifty-three.

Twenty periods: the round cap is what made the plan possible. The twenty-period row deserves a note because an earlier revision of this work reported that instance as unsolved, and the difference is one parameter. Under an uncapped cut-and-repack walk the construction spent 2 224 s of a 3 065 s run inside the partition — 387 cut rounds over the first fourteen of forty (direction, period) pairs, 8 709 placeability calls — and reached the deadline with no plan at all. Step 3's deliverable is columns and not the partition, which the cover of Section 2.8.1 already conceded, so a pair whose parts will not place need not be pursued round after round: with the rounds capped at two, the parts that did place kept as columns and the failing pairs left to step 4, the same instance returns a verified one-hundred-and-sixty-truck plan at 230 594.36, the root construction finding it at 1 418 s. Forty-four of the eighty-seven (direction, period) pairs the construction attempted were left to step 4 in that run. The cap is a parameter of a

Table 11: The thirty-six generated instances through MATH-3D-SCPTOP (Section 2.8.1 inside the fleet tree of Section 2.9), every row on one commit (`0c0c16c`), a 900 s budget, ten threads, a 20 GB per-model cap and seed 0, run sequentially so that no two rows shared the cores. Wall times run from 603.1 s to 900.1 s; the longest row exceeds the budget by 0.1 s, because every phase of the construction carries a floor so that a phase begun just before the wall still gets a token solve. *ver* is the number of dispatched trucks that an independent verifier re-derives from the column's own fill and finds legal — placement, stack height, crush limit, payload and the utilisation floor — out of the number dispatched; no plan in this table carries a complaint. LB is the certified volumetric relaxation, valid for the 3D problem by Proposition 2, and is printed only where the sweep came back with every pair proved; *gap* is to that bound and is an optimality gap only to the extent the bound is tight, which Section 3.7 shows it is not where the crush limit binds. *parts* is the number of distinct truck compositions the partition proposed, of which *unpl.* were *proved* unplaceable and *und.* left undecided; an undecided composition is counted on neither side. The last two columns are the volumetric benchmark SCPTOP on the same instance: the loads it dispatches, and how many of them a constraint programme proves cannot be placed in a trailer at all. The last column reports the loads that constraint programme left *undecided*, which must be read beside the previous one: a low count of proven infeasibilities means nothing on a row where most loads were undecided. The two *und.* columns are different quantities — one counts compositions the partition proposed, the other loads SCPTOP dispatched — and the group headers say which is which.

				MATH-3D-SCPTOP						partition			SCPTOP loads		
Instance	$\|N\|$	$\|T\|$	$\|R\|$	z	trucks	ver	LB (cert.)	gap	nodes	parts	unpl.	und.	loads	unpl.	und.
I_4_10_2_120	4	10	2	62 505.08	45	45 / 45	53 425.21	14.53%	0	214	64	9	36	30	5
I_4_10_2_60	4	10	2	*no plan*	—	—	31 847.48	—	0	64	32	0	22	20	1
I_4_10_2_90	4	10	2	45 230.80	29	29 / 29	42 208.62	6.68%	3	282	82	4	26	22	3
I_4_10_3_120	4	10	3	57 953.00	42	42 / 42	47 690.39	17.71%	1	203	59	8	32	26	6
I_4_10_3_60	4	10	3	34 236.78	23	23 / 23	30 862.57	9.86%	2	127	42	13	20	18	2
I_4_10_3_90	4	10	3	44 485.13	30	30 / 30	44 456.13	0.07%	16	543	88	25	31	6	19
I_4_10_4_120	4	10	4	50 734.95	35	35 / 35	50 555.71	0.35%	4	450	133	18	35	12	21
I_4_10_4_60	4	10	4	*no plan*	—	—	35 872.62	—	0	51	32	0	24	23	0
I_4_10_4_90	4	10	4	43 698.86	29	29 / 29	40 487.62	7.35%	1	224	68	10	26	17	8
I_4_5_2_120	4	5	2	27 226.24	18	18 / 18	25 172.69	7.54%	10	490	242	22	16	16	0
I_4_5_2_60	4	5	2	14 715.58	9	9 / 9	13 768.49	6.44%	76	376	25	56	8	6	1
I_4_5_2_90	4	5	2	19 740.03	13	13 / 13	17 763.70	10.01%	126	2430	1193	12	11	9	2
I_4_5_3_120	4	5	3	25 925.74	16	16 / 16	25 787.76	0.53%	11	463	191	45	16	6	8
I_4_5_3_60	4	5	3	19 669.36	15	15 / 15	13 626.60	30.72%	1	75	30	0	9	9	0
I_4_5_3_90	4	5	3	21 502.06	14	14 / 14	21 498.70	0.02%	51	138	0	18	14	0	10
I_4_5_4_120	4	5	4	28 229.12	17	17 / 17	26 289.27	6.87%	19	937	426	38	15	9	5
I_4_5_4_60	4	5	4	16 166.20	12	12 / 12	12 969.54	19.77%	8	228	126	12	9	8	0
I_4_5_4_90	4	5	4	21 531.74	14	14 / 14	20 525.49	4.67%	19	534	273	10	13	7	5
I_8_10_2_120	8	10	2	63 265.31	44	44 / 44	55 483.16	12.30%	1	462	122	2	37	30	6
I_8_10_2_60	8	10	2	*no plan*	—	—	30 630.70	—	0	88	48	0	22	21	0
I_8_10_2_90	8	10	2	50 008.13	36	36 / 36	43 242.73	13.53%	1	210	56	4	30	27	1
I_8_10_3_120	8	10	3	58 632.27	40	40 / 40	52 387.71	10.65%	1	409	102	11	35	24	7
I_8_10_3_60	8	10	3	33 286.65	24	24 / 24	*unproved*	—	1	170	45	5	20	17	1
I_8_10_3_90	8	10	3	41 501.25	27	27 / 27	37 630.17	9.33%	1	112	44	9	24	21	2
I_8_10_4_120	8	10	4	66 209.55	44	44 / 44	58 184.10	12.12%	1	276	83	5	37	30	7
I_8_10_4_60	8	10	4	*no plan*	—	—	30 093.95	—	0	51	28	1	21	19	0
I_8_10_4_90	8	10	4	46 474.74	29	29 / 29	43 174.67	7.10%	1	285	84	4	27	15	11
I_8_5_2_120	8	5	2	26 181.24	16	16 / 16	26 066.68	0.44%	12	501	168	0	16	5	7
I_8_5_2_60	8	5	2	20 422.01	15	15 / 15	15 159.80	25.77%	3	211	89	0	10	9	0
I_8_5_2_90	8	5	2	18 796.96	12	12 / 12	18 795.54	0.01%	68	35	0	1	12	0	12
I_8_5_3_120	8	5	3	22 162.16	14	14 / 14	21 045.66	5.04%	2	258	70	24	13	9	3
I_8_5_3_60	8	5	3	*no plan*	—	—	14 101.19	—	0	53	35	1	9	9	0
I_8_5_3_90	8	5	3	18 649.29	12	12 / 12	18 648.12	0.01%	80	98	5	9	12	0	11
I_8_5_4_120	8	5	4	24 423.97	15	15 / 15	24 338.21	0.35%	8	361	85	18	15	5	7
I_8_5_4_60	8	5	4	17 693.47	12	12 / 12	13 578.09	23.26%	2	62	26	1	8	8	0
I_8_5_4_90	8	5	4	21 192.52	13	13 / 13	20 993.24	0.94%	6	395	131	2	13	8	3

Table 12: Table 11 by instance factor. *plans* counts the instances on which MATH-3D-SCPTOP returned a verified plan inside the 900 s budget; the mean gap is over those instances only and over the rows whose bound was proved, and the count of each is given so that the denominator is never in doubt. The last three columns are the volumetric benchmark's dispatched loads: the share a constraint programme *proves* unplaceable, the share it leaves *undecided*, and the share it places. The undecided share must be read with the other two: a low count of proven infeasibilities means nothing where most loads were undecided.

factor	value	plans	mean gap (rows)	loads	unpl.	und.	placed
$\|N\|$	4	16 / 18	8.9% (16)	363	67.2%	26.4%	6.3%
	8	15 / 18	8.6% (14)	361	71.2%	21.6%	7.2%
$\|T\|$	5	17 / 18	8.4% (17)	219	56.2%	33.8%	10.0%
	10	14 / 18	9.4% (13)	505	74.9%	19.8%	5.3%
$\|R\|$	2	10 / 12	9.7% (10)	246	79.3%	15.4%	5.3%
	3	11 / 12	8.4% (10)	235	61.7%	29.4%	8.9%
	4	10 / 12	8.3% (10)	243	66.3%	27.6%	6.2%
v	60	7 / 12	19.3% (6)	182	91.8%	2.7%	5.5%
	90	12 / 12	5.0% (12)	239	55.2%	36.4%	8.4%
	120	12 / 12	7.4% (12)	303	66.7%	27.1%	6.3%
all		31 / 36	8.8% (30)	724	69.2%	24.0%	6.8%

primal heuristic, so this is a statement about the heuristic and not about the problem.

The full factorial. Tables 11 and 12 run the same method over the thirty-six generated instances at a uniform 900 s, one commit, sequentially. It returns a verified plan on thirty-one of thirty-six and a proved bound on thirty-five; no plan carries a verification complaint. Wall times run from 603.1 s to 900.1 s, so the longest row overruns the 900 s budget by 0.1 s: every phase of the construction carries a floor, so a phase begun just before the wall still gets a token solve. The mean certified gap over the thirty rows that have both a plan and a proved bound is 8.8%, and the largest is 30.72%.

The five failures are the informative part, and they are not spread across the factorial: all five have $v = 60$, the coarsest granularity, where RTIs are largest and a single container is the largest fraction of a trailer. At $v = 60$ the method returns a plan on seven of twelve instances, and the mean gap of 19.3% is over the six of those seven whose bound was also proved ($I_8_10_3_60$ has a plan and an unproved bound, so it carries no gap); at $v = 90$ and $v = 120$ it returns a plan on twelve of twelve with mean gaps of 5.0% and 7.4% over twelve rows each. Table 12 prints each denominator beside its mean. The same granularity is where the volumetric benchmark's loads are least placeable (91.8% proved unplaceable, against 55.2% and 66.7%), so the two difficulties have a common cause: when one container is a large fraction of the trailer, the integer structure of the loading problem is coarse, the volumetric relaxation is at its least informative, and the construction has the least room to cover a delivery vector with full trucks. A plan is not returned rather than an invalid one being returned, which is a consequence of the formulation: a column that the verifier would reject never enters the pool.

What the tree contributes, and where. The contribution is uneven across the ladder and is not averaged away. At five periods the tree opens 67 nodes and improves nothing: the root construction returns 18 796.96 at 140.2 s and that is the plan. At ten periods it opens 68

and seven incumbent improvements take 38 865.30 at 135.9 s to 38 855.34 at 590.7 s, a gain of 0.026%. At twelve periods it opens 11 and five improvements take 53 195.84 at 459.0 s to 52 129.45 at 1 648.5 s, a gain of 2.005% — this is where branching does the most work in the paper. At fifteen and twenty it opens 1 and 2 nodes and the reported plan is the root construction's. Over the thirty-six generated instances the node counts range from 0 to 126; they are not a predictor of the gap, which $I_4_5_2_90$ makes plain at 126 nodes and 10.0%.

Repeat runs matter at these margins, and the exclusion is stated rather than applied. On $I_12_12_2_90_k25$ two runs of the same knob set on two commits returned 52 129.45 and 52 127.89, both with thirty-four trucks, a spread of 0.003%; two further runs under a delivery-branching variant since withdrawn for a diagnosed defect returned 53 005.78 and 53 425.07, both with thirty-five trucks. The method is deadline-driven at every phase, so a step-2 solve landing at 299 s rather than 301 s takes a different branch; no objective delta below a few hundredths of a percent on an instance of this kind is reported here without its measured spread beside it.

Where the time goes. It depends on whether the crush limit binds, and the split is read from each run's own clock. On $I_8_5_2_90$ column generation takes 864 s of an 873 s run and everything else 8 s: the geometry is free and the budget goes into pricing columns that do not improve the plan. On $I_8_10_2_90_k25$ the aggregate master of step 2 takes 573 s of 883, column generation 181 s, the partitions 79 s of which 65 s are placeability calls, and the integer master 35 s. On $I_12_12_2_90_k25$ the integer master of step 4 takes 1 024 s of 1 761 and the partitions 325 s. On the two crush-binding rungs the partition dominates: 934 s of 1 746 on $I_16_15_2_90_k25$ and 1 677 s of 3 492 on $I_24_20_2_90_k50$, of which 369 s and 1 246 s are placeability calls. The aggregate and integer masters are the scaling limit where the crush limit does not bind, and they are shared with the volumetric method; where it does bind, proving parts unplaceable is. Within placeability, the constructive shelf design answers essentially every positive verdict and the constraint programme supplies the proofs of impossibility: over the thirty-six generated instances the shelf design placed 24 686 loads and the constraint programme 246, while the programme proved 4 327 of the 11 866 distinct compositions proposed unplaceable and left 397 undecided.

The pool and the pricing ladder. Tables 8 and 9 isolate two decisions. The pure single-commodity pool costs nothing on $I_8_5_2_90$ once step 2 counts stacks in integers and the region $X \leq X^\star - 1$ is constructed first: the method returns the twelve-truck plan at 18 796.96, to the cent the value of the hand-seeded reference that was removed from the method. Without those two decisions it returned thirteen trucks at 19 682.36, for the reason given in Section 2.8.1.

The heuristic rung is isolated by a three-arm experiment from one pure pool — the exact oracle alone, the rung pre-empting the oracle wherever it qualifies, and the two additively — run in two configurations that answer two different questions. At 5 s per call the exact oracle is invoked at every pair in *all three* arms, so that the arms differ only in which columns enter the pool and the walls differ only by the rung's own cost; at 20 s, the campaign's own per-call budget, the arms run the production structure, in which a pre-empted oracle is not called at all, so those walls are comparable. What the rung buys is **LP quality at equal work**, and it buys it only where the instance is large enough to need balanced loads.

On $I_8_5_2_90$ all three arms land within 0.003% of one another after five sweeps — 18 796.61 exact-only, 18 796.04 with the rung pre-empting, 18 796.11 with both — and the rung contributes no truck to the reported plan: there it is inert, not harmful. On $I_16_15_2_90_k25$ the separation is wide. At 20 s and six sweeps, exact-only reaches 76 103.92 in 744.5 s, the rung-fed

arm reaches 75 983.75 in 405.6 s — a better value 1.8× sooner — and running both generators reaches 75 361.05. At matched wall within that configuration the rung-fed arm is 2.0% tighter at ∼235 s and 1.6% at ∼410 s. The two configurations ran at 1.96 and ∼8 sampled cores, so no figure of one is compared with a figure of the other.

One claim this paper made in an earlier revision is withdrawn here. At 5 s per call the exact oracle alone leaves 99.6 units of artificials in this master after three sweeps while one rung-fed sweep clears them, and that was reported as a structural limit of exact pricing. It is not one: at 20 s the exact arm reaches zero artificials at *sweep one*, with an LP of 78 746.59, and finishes feasible. The infeasible master was an artefact of a five-second cap, and the case for the rung rests on LP quality at equal work and not on feasibility. What remains true, and is a different statement, is that the pure pool of seventeen single-commodity trucks *by itself* cannot cover demand within the fleet: the master over that pool alone carries $1.55 \cdot 10^8$ of artificials, and some generator has to clear them.

The two generators are complementary rather than alternative, which is why the best of the three arms is the one that runs both. Within that arm, where both generators are called at every pair and so compared under one dual trajectory, the oracle's columns carry 4.72 distinct commodities (median 5) with 0.522 of the load in the heaviest and the rung's carry 10.17 (median 14 of 16) at 0.374; pricing maximises a linear objective and so returns extreme, concentrated loads, and a balanced load is not an extreme point of $\mathrm{conv}(\Omega^g)$ at any duals. Which arrangement to run is a budget question rather than a choice between the two, and Section 2.6 states the condition: both while the mean per-pair oracle time is at or under 5 s, the rung pre-empting above it. At 20 s that is pre-emption, which is what the code already did, so every row of this section was produced by the algorithm just described.

In the one-commit runs the rung served 13, 15, 66, 206 and 318 of the 215, 69, 227, 585 and 873 pricing calls, and the trucks it contributed to the reported plan are none, none, none, 44 of 54 and 122 of 160: nothing on the three instances whose certified gap is under 0.05%, and most of the fleet on the two whose catalogue makes the crush limit bind, which are also the two on which the pure pool alone cannot cover demand (Table 9). It costs 18.9 s of the 405.6 s arm, 4.7%. The certificate is never the rung's: the bound of every row is one uncapped exact sweep of the volumetric oracle, and a sweep at which the rung served any pair cannot report *proved*.

3.6 Against the procedure in use

The comparator for the 3D rows is the company procedure, because Section 3.5 established that the compact 3D model cannot serve as one. **That comparison is not reported in this revision.** The reason is stated here rather than deferred, because the file that would fill the table exists and must not be read as filling it.

> Not available in this revision. The case study comparing MATH-3D-SCPTOP against PLC-SCPTOP on the nine company instances requires a run of the method of Section 2 against baselines solved to the gap the comparison claims. Neither condition is met by the recorded run on disk, for two independent reasons given below, so no cost, truck count or percentage saving is quoted against the company procedure anywhere in this paper.

First, the method column of that run was produced by the earlier branch-and-price configuration of Section 3.3 and not by the method of Section 2 — a pool seeded with the company's own configurations, and the integer master rather than the flow-first relax and fix as the primal. Labelling it with the method's name would be the error this section is organised to avoid.

Second, and independently, all nine company baselines stopped on their time limit rather than at the requested MIP gap of zero. The evaluation solve is given 315 s of the instance's 900 s budget and every one of the nine returned TIME_LIMIT; the stage's recorded wall times, 513.8 to 1 493.6 s, are larger because the stage also rebuilds the configuration pool in up to four rounds of 180 s before the evaluation solve, and three instances ran concurrently. A saving measured over a baseline that was not solved is a saving over whatever the baseline's solver happened to hold when the clock ran out; it bounds the true saving from one side only, and in the wrong direction for the claim, since a better-solved baseline can only reduce it. The same run makes the point against itself: its nine savings over the company procedure span −1.25% to +18.47% and are *negative* on two instances, which is not a result that survives either defect being repaired in an unknown direction.

Two observations from the same run do *not* depend on the solution procedure and are reported, because they are properties of the company's configuration set rather than of anything the method did. Reconstructing that set from the planner's average mixes yields very few distinct configurations when demand is stable, and on three of the nine instances the reconstruction had to be extended by 77, 66 and 73 configurations before the programme could be served at all; on the other six it needed none. And on eight of the nine instances the *dedicated* reading of a configuration — one per product, rather than shared across an RTI family — served the programme better than the shared one. Both say the same thing about the practical cost of a fixed configuration set: it is not that the procedure fails, since the plant plainly operates, but that keeping it feasible as the programme moves requires a planner to maintain a large and growing catalogue by hand, every entry of which was found by trial rather than by design.

The nine instances also carry the one piece of evidence about real containers that does not depend on the method at all. The volumetric benchmark, run on them under the same budgets, dispatches 392 truck loads of which 169 — 43.1% — are *proved* unplaceable by the same constraint programme used throughout. That run did not record how many of the remainder were left undecided, so 43.1% is a lower bound on the share that cannot be placed and not an estimate of it, and it is not comparable with the 69.2% of Section 3.7, which is measured over the generated instances and reports its undecided share.

3.7 Is a volumetric capacity adequate?

This is the question that decides whether the 3D representation buys anything, and it is answered by proof rather than by argument: every truck load the volumetric benchmark dispatches is submitted to a constraint programme asking whether that exact set of RTIs can be stacked and placed in a trailer at all, with the true slot bound (48) in force so that an infeasibility verdict is a proof and not a modelling artefact.

The census. Over the thirty-six generated instances the volumetric benchmark dispatches 724 truck loads. Of those, 501 (69.2%) are *proved* unplaceable, 49 (6.8%) are proved placeable, and 174 (24.0%) are left undecided within the limit. The undecided share must be read with the other two, and it is not noise: a constraint programme cannot decide easy loads quickly either, and the last column of Table 11 reports the count per instance — not to be confused with the *und.* column under *partition*, which counts compositions the construction proposed rather than loads SCPTOP dispatched. What the census supports is therefore a statement about the loads it *decided*: of the 550 loads decided either way, 91.1% cannot be placed.

The error has a direction, and the direction is what matters operationally. A volumetric capacity ignores the remainders that footprints and layer heights leave unusable, so where it

Table 13: **What volumetric planning costs, constructed rather than estimated.** Trucks dispatched, on the two instances where the per-stack crush limit binds. Row 1 is the volumetric benchmark's own plan, which is not executable: 41 of its 45 dispatched loads on the first instance and 119 of its 137 on the second are *proved* unplaceable, with *none* left undecided on either, so those two rows are fully decided and the shares — 91.1% and 86.9% — carry no undecided caveat. Row 2 splits each unplaceable load where it stands, with no consolidation; it is reported for contrast and is *not* the comparison, because consolidation is worth twelve and forty-four trucks. Row 3 is the comparison: the volumetric plan's delivery quantities kept exactly, each period re-packed with the same partition-and-pack machinery the method uses, every resulting truck verified. Row 4 is MATH-3D-SCPTOP, which also chooses the quantities. The volumetric plan on $I_24_20_2_90_k50$ is a *memory stop* and not an optimum (200 266.21, Table 5), so a better volumetric plan might repair into fewer than 212 trucks and that row's margin is overstated by an unquantified amount in the method's favour; the $I_16_15_2_90_k25$ plan is the proved volumetric optimum and carries no such caveat.

	$I_16_15_2_90_k25$	$I_24_20_2_90_k50$
volumetric plan (not executable)	45	137
repaired load by load (contrast only)	86	256
re-packed, quantities kept	**74**	**212**
MATH-3D-SCPTOP (which also chooses the quantities)	**54**	**160**

errs it errs by dispatching loads that do not fit. A planner handed such a plan discovers the shortfall on the loading dock, not in the model. By granularity the shares are 91.8% proved unplaceable at $v = 60$ (with only 2.7% undecided, so that row is nearly fully decided), 55.2% at $v = 90$ and 66.7% at $v = 120$.

The two ladder instances on which the crush limit binds give the same finding with no undecided caveat at all, because there the oracle decided every load. On $I_16_15_2_90_k25$, where the volumetric plan is the *proved* optimum, 41 of its 45 dispatched loads are proved unplaceable and four are placeable, none undecided; on $I_24_20_2_90_k50$ it is 119 of 137, again none undecided. A volumetric optimum can therefore be a plan nine tenths of which cannot be loaded, and that statement rests on a complete audit rather than on a share of the decided.

What volumetric planning costs, constructed rather than estimated. The census says a volumetric plan's loads frequently cannot be placed; it does not say what that costs, and the cost is the answer to the third question of this section. To answer that, each volumetric plan is repaired into an executable one on the two instances where the crush limit binds. The *competent* repair keeps the volumetric plan's delivery quantities unchanged and re-packs each period's freight with the same partition-and-pack machinery the method itself uses; every resulting truck is verified. A weaker, load-local repair — splitting each unplaceable load where it stands, with no consolidation — is reported for contrast and is *not* used as the comparison, because consolidation is worth twelve and forty-four trucks and a planner would do it. The outcome is Table 13, at the head of this section.

Against the re-packed baseline the method uses 27.0% and 24.5% fewer trucks and costs 19.1% and 16.2% less, with no pair left unpacked and no truck failing verification in either case; the denominator of every one of those four percentages is the re-packed baseline. The difference between 74 and 54 is what optimising the delivery quantities against the geometry buys, over and above packing the same quantities well. Two qualifications. The re-pack solves a cover, so it ships 80 and 137 units beyond requirement, and pricing only its trucks therefore

understates its cost — the margins are conservative in the method's favour only in the sense that they are lower bounds on the true difference. And the repaired plan is a *baseline, not a bound*: on $I_8_5_2_90$ the load-local repair needs thirteen trucks where twelve is the proved optimum, because repairing a fixed assignment of freight to trucks is not the same as planning afresh.

A harder volumetric baseline, and what is not yet known about it. The comparison above is against a *plain* volumetric model, and the obvious objection is that a planner who had read this section would not build one: the mechanism is computable from the catalogue, so the capacity could be deflated to reflect it. The honest answer is that a step in that direction has been measured and does not close the gap, and that the rest of it has not been measured at all. Adding to the compact volumetric model the stack-count rows of Section 2.10 — an integer number of stacks per footprint, each within the crush limit and the height budget, their floor areas summed against the trailer floor, which is a valid inequality for every placeable load — raises its answer on $I_16_15_2_90_k25$ from forty-five trucks to forty-eight, at 71 072.06 with a dual bound of 70 435.04 after 600 s, and raises the dual bound of the relaxation by 2 303, about a fifth of the gap the plain bound leaves. On $I_12_12_2_90_k25$, where the crush limit does not bind, the same rows change the truck count not at all.

Two things follow, and we draw only those. The rows are a real tightening of the *bound* and are named in the conclusions as the next step for it. But forty-eight trucks is still eleven percent below the method's verified fifty-four, and *whether those forty-eight loads can be placed has not been tested*: the stack rows are necessary for placeability, not sufficient, since they say nothing about how the stacks tile the floor. Until that test is run, the stack-aware volumetric model is not a baseline this paper can report a saving over, and claiming one would repeat exactly the error the plain volumetric model is being faulted for.

The mechanism behind all of it. Both the census and the cost above have one cause, and it is worth isolating because it is the part a planner can check in advance. The error of a volumetric capacity is governed by how much it over-credits a truck relative to what physically fits, and the clearest single case is measured directly. On $I_16_15_2_90_k25$ the payload admits 57 full RTIs of product P_1 in one truck, and 32 is the largest number that can be placed — a proved bound, obtained by running the pricing problem for that commodity alone to optimality. The cause is that the family's container is 650 mm tall, so four layers fit under the 3 000 mm roof, while at 378 to 484 kg full only two fit under the 1 200 kg crush limit. A volumetric fill therefore credits a truck with roughly twice the placeable count of that family. The same divergence is why the certified bound on the two crush-binding rungs of Table 6 sits 13.3% below the best verified plan: the bound credits every such stack with four layers where two can be carried, and we attribute the gap to the bound rather than to the plan on that evidence — noting that nothing in this paper proves it, and that the fifty-four- and one-hundred-and-sixty-truck plans are reported as what they are, verified upper bounds under a stated budget.

The layer ratio alone does not predict it, and we say so. It is tempting to turn the mechanism into a one-number test, and the obvious candidate fails. The sweep is eighteen generated instances: three seeds crossed with six dose levels, where the dose is a scaling of the product weights alone, so that within a seed the footprints, heights and demands are identical across the six levels and only the weights move. Over those eighteen, the share of a volumetric plan's own dispatched loads that a placeability proof rejects correlates at only 0.42 with the

ratio of height-limited to crush-limited layer counts. Three of the eighteen volumetric solves stopped on their time limit rather than at optimality, which affects which plan was audited but not the audit. The sweep contains three instances at 0% with that ratio between 0.98 and 1.25, and one at 90.9% with the ratio at 1.08. The ratio is part of the mechanism and is not a predictor of the share, and a predictor validated across instances is not established here. What *is* established is narrower, and it is a one-sided statement.

Define, for one container type and the trailer of Section 3.2, the *analytic credit ratio* θ: the numerator is the count a volumetric capacity credits a truck with, $\min\{\Gamma_K/\gamma_{pr},\, pK^w/\bar{w}_r\}$, and the denominator is an *analytic* count of how many can be carried, $\min\{s_r{\cdot}\min(L_r^h, L_r^c),\, pK^w/\bar{w}_r\}$. Here $\bar{w}_r$ is the catalogued gross weight of one full RTI of type r, s_r the number of its footprints that a single orientation tiles onto the trailer floor, and $L_r^h = \lfloor rK/r_{pr} \rfloor$, $L_r^c = \lfloor W^{\max}/\bar{w}_r \rfloor$ its height-limited and crush-limited layer counts. Both ends obey the same payload: a grid count no truck can carry is not a count of what can be carried, and the earlier revision of this paper divided a payload-capped numerator by an uncapped denominator, which reported a spurious under-credit on eight of the fifteen types — Container 9's 136 "placeable" RTIs weigh 74.8 t against a 24 t cap. The denominator is also analytic and not proved: it is a grid count in a single orientation with *no* packing oracle run, so it is an *upper* estimate of what fits, and $\theta > 1$ is correspondingly a conservative diagnosis of over-credit.

Over the fifteen container types of the industrial catalogue θ has median 1.000, ranges from 1.000 to 2.000, and exceeds 1.20 on four types. No type has $\theta < 1$. Read correctly that says two things, and the first is stronger than the statement this revision replaces. On this catalogue a volumetric capacity is *never* conservative: there is no real type it under-credits, so the error cannot run in the planner's favour. Nine of the fifteen sit exactly at the analytic count, and for a reason worth naming: on eight of those nine the 24 t payload binds *both* ends, so the volumetric capacity is exact there rather than merely safe, and the ninth is a pallet-sized container that tiles the trailer floor so closely that its volume count and its grid count coincide at 22. And four of fifteen are over-credited by more than a fifth, which is enough for a plan drawn over a mixed catalogue to contain unplaceable loads, since a plan fails on its worst container and not on its median one.

The instances on which the census above was taken are *generated*, with θ in a range the generator controls, and the catalogue measurement is a realism check on that range rather than a second census. Three statements therefore survive and a fourth does not. It survives that a volumetric capacity can err badly and that where it errs it errs by prescribing loads that cannot be placed — the census proves that on the generated set, and the mechanism explains it. It survives that θ is computable from the catalogue and the trailer alone, with no model solved, so a planner can tell in advance which regime a catalogue is in. It survives, and is new here, that on the industrial catalogue the error is one-sided: the generated sweep's range 0.83 to 1.44 reaches below 1 where the real catalogue never does, so treating a volumetric capacity as a conservative simplification is not defensible on this catalogue in particular. What does *not* survive is any transfer of the census *share* to the real catalogue: $\theta \geq 1$ on every type says the error never runs the other way, not what fraction of a real plan's loads would fail, and a study that wanted that number would have to take its census on real catalogues and not on generated ones.

3.8 What the bounds and the claims rest on

Four statements belong with the results rather than with the claims, and they are made here so that every number in this section is read correctly.

No optimum is proved by the method. Optimality is claimed in this paper exactly once, on $I_8_5_2_90$, and it is established by inclusion from an exactly solved relaxation and not by the method's own certification (Section 3.5). In every run reported here the global bound equals the root bound, because a region's certificate is taken from the volumetric relaxation and no such certificate can exceed the relaxation's own optimum; where that optimum coincides with the 3D optimum the method can prove optimality by inclusion, and where the crush limit binds and it does not, the residual gap is not closable by more search. Regions remain open on every row of Table 6 but the first, and a run with open regions has not searched the space whatever its gap.

What the bound costs and what the alternatives return. Certifying the volumetric relaxation is cheap and is the reason the division of labour of Section 2.5 is affordable: over the forty-one runs reported here the sweep returns in 2.5 to 111.8 s and comes back with every pair proved on forty of them, the exception being the one row of Table 11 that prints *unproved*. The geometric sweep is the other extreme: it certified no region on any run in this study, and the three leaves of $I_8_5_2_90$ that the search could not refute took 237 s each to attempt and returned nothing proved. The Lagrangian bound (60) sits between them and is reported rather than relied on, for the reason given when it was stated: on $I_8_5_2_90$, whose optimum is 18 796.96, it returns 18 778.31 — valid, and beaten by the volumetric certificate's 18 795.54, because ten proved pairs at $|K| = 12$ accumulate 120 units of penalty against a target of 1.42.

A lower bound is printed only where a sweep proved it. Where no full sweep completed with every pair proved, the value the run records is the restricted master's, which is an *upper* bound on the master LP, and it is never printed as a bound. One row of Table 11 is in that position and says *unproved*; it is excluded from the mean gap, and the count of rows behind every mean gap in Table 12 is printed beside it for that reason.

The compact 3D model's bound is not a bound on the problem. It restricts each direction a priori to a catalogue of $n_{\mathrm{LC}} = 3$ layouts shared by all its trucks, where Section 1 lets every truck design its own. A restriction's optimum is at or above the problem's, so its dual bound bounds that restriction and not $z^\star$. Its *plan* is used as a comparator and its bound never as an end of an interval. In particular the compact model and the method do *not* bracket the optimum on $I_8_5_2_90$; the interval that holds $z^\star$ there is [18 796.96, 18 796.96], and it comes from the volumetric relaxation.

Every reported plan is verified, and the verifier is independent. Every plan in Tables 11 and 6–10 is re-derived truck by truck from each column's own fill and checked against (49)–(54) — geometry, stack heights, the per-slot crush limit, the payload and the utilisation floor — by a routine independent of the search and of the oracle that built the column; no plan reported here carries a complaint. That is a property of the formulation rather than of the search: a column is a witnessed layout, so a plan assembled from columns is executable by construction and can be handed to a loading crew as a drawing.

Two things about its independence should be stated precisely rather than overstated. First, it is independent of the *producer* and not disjoint from the implementation: the predicate that decides whether a truck is full on the binding resource is the same one the oracle, the column reductions and the partition floor use, as are the lookups from a commodity to its weight and to its RTI type. A defect in the *definition* of a full truck would therefore not be caught here; a defect in a solver's *satisfaction* of it would, and that is the failure mode the check exists

for. Second, the per-slot crush limit is re-derived from the fill the slot carries, and the routine complains if that fill does not describe the stack it is attached to — same RTI types, same layer counts — so a load relabelled onto an old layout is rejected rather than passed. Only a slot carrying no fill at all falls back on the weight stored with the column, which is the family maximum and therefore the conservative reading for a *cap*; it would not be sound for a floor, and no floor is checked that way.

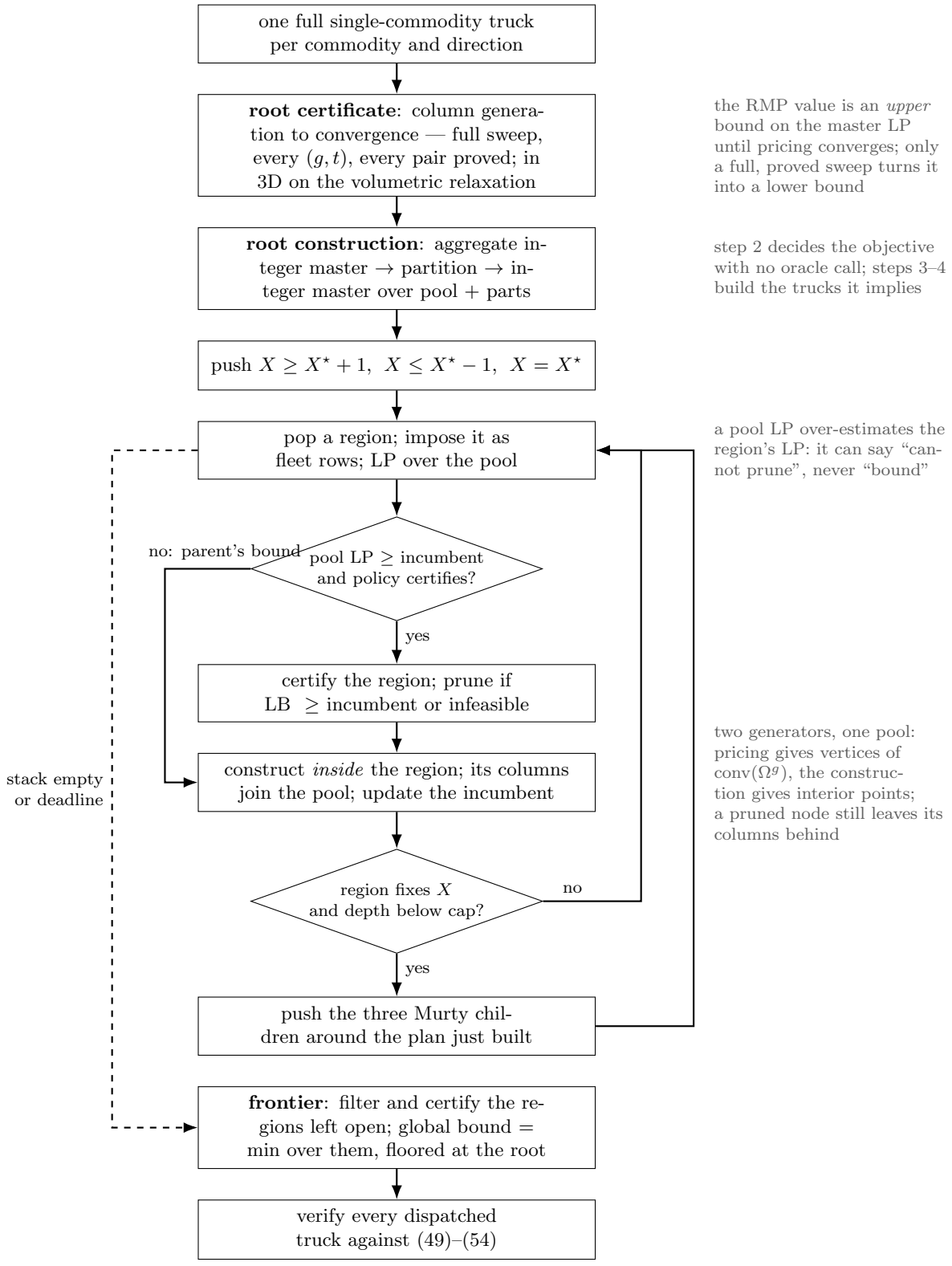


Figure 6: The fleet tree end to end. The search's value is not that it reaches an integral relaxation — it essentially never does — but that each region asks the construction for a plan under constraints the root never saw, and the loads that plan needs are added to a pool every later node can use. The bound is decided by certification alone: a region that was not swept in full, with every pair proved, carries its parent's bound, and the global bound is the minimum over the frontier rather than the root value.

Algorithm 1 The fleet tree for 3D-SCPTOP: branch and bound over the fleet vector with the flow-first construction as primal

$\bar{\Omega} \leftarrow$ one full single-commodity truck per commodity and direction ▷ Section 2.7
$\mathrm{LB}_0 \leftarrow \textsc{Certify}(\bar{\Omega},\ \text{root})$ ▷ full sweep, every (g,t), every pair proved; else no bound. In 3D: on the volumetric relaxation, Prop. 2
$(\bar{z}, \mathbf{n}^\star, \bar{\Omega}) \leftarrow \textsc{Construct}(\bar{\Omega},\ \text{root})$; $X^\star \leftarrow \sum_{g,t} n^\star_{gt}$ ▷ steps 1–4, Section 2.8.1
$\mathcal{S} \leftarrow \left[\, X \ge X^\star + 1,\ \ X = X^\star,\ \ X \le X^\star - 1 \,\right]$ ▷ a stack; the region that can beat the incumbent is expanded first
while $\mathcal{S} \neq \emptyset$ and time remains **do**
    $\mathcal{R} \leftarrow$ pop $\mathcal{S}$; impose $\mathcal{R}$ on the RMP as fleet rows
    $\tilde{z} \leftarrow$ LP over $\bar{\Omega}$ inside $\mathcal{R}$ ▷ over-estimates the region's LP: a filter, never a bound
    if LP infeasible **then**
        continue ▷ discarded
    end if
    if $\tilde{z} \ge \bar{z}$ and the policy certifies $\mathcal{R}$ **then**
        $(\mathrm{LB}_{\mathcal{R}}, \text{proved}) \leftarrow \textsc{Certify}(\bar{\Omega}, \mathcal{R})$
        if region infeasible or $\mathrm{LB}_{\mathcal{R}} \ge \bar{z}$ **then**
            continue ▷ pruned
        end if
        record $\mathrm{LB}_{\mathcal{R}}$ as an open bound
    else
        $\mathcal{R}$ carries its parent's bound
    end if
    $(z, \hat{\mathbf{n}}, \bar{\Omega}) \leftarrow \textsc{Construct}(\bar{\Omega},\ \mathcal{R})$ ▷ inside the region; its columns join the shared pool; $\hat{\mathbf{n}}$ is the plan just constructed
    if no plan **then**
        price under the region's duals; Construct once more; **continue** if still none
    end if
    if $z < \bar{z}$ **then**
        $(\bar{z}, \mathbf{n}^\star) \leftarrow (z, \hat{\mathbf{n}})$ ▷ new incumbent
    end if
    if $\mathcal{R}$ does not fix X **then**
        push $\mathcal{R} \cap \{X \ge X_2 + 1\}$, $\mathcal{R} \cap \{X = X_2\}$, $\mathcal{R} \cap \{X \le X_2 - 1\}$ with $X_2 = \sum_{g,t} \hat{n}_{gt}$ ▷ level 1 recurses
    end if
    if $\mathcal{R}$ fixes X and its depth j is below the cap **then**
        $(g,t) \leftarrow$ the j-th pair in the fixed order
        push $\mathcal{R} \cap \{n_{gt} \ge \hat{n}_{gt} + 1\}$, $\mathcal{R} \cap \{n_{gt} \le \hat{n}_{gt} - 1\}$, $\mathcal{R} \cap \{n_{gt} = \hat{n}_{gt}\}$ onto $\mathcal{S}$ ▷ Murty, around the plan just constructed: n_{gt} is the region's variable, $\hat{n}_{gt}$ its value in that plan
    end if
    release $\mathcal{R}$
end while
for all $\mathcal{R}$ still on $\mathcal{S}$, while a reserve of time remains **do**
    filter by the pool LP; certify the survivors; record their bounds ▷ the frontier
end for
$\mathrm{LB} \leftarrow \max\{\mathrm{LB}_0,\ \min \text{ over open and frontier bounds}\}$, capped at $\bar{z}$
verify every dispatched truck of the incumbent against (49)–(54)
return $(\bar{z}, \mathrm{LB})$; optimality is claimed only if LB_0 was proved, $\mathcal{S} = \emptyset$, no restart occurred and $\bar{z} = \mathrm{LB}$